

JMPM

Changelog

DATA
05/09/2026

ARQUIVO
CHANGELOG.md

Todas as mudanças notáveis deste projeto são documentadas aqui.

[3.0.5] — 2026-09-05

Alterado

- **Toolchain Go 1.24.2 → 1.27.1** (`go.mod` : `go 1.27.0` / `toolchain go1.27.1` ; workflows do GitHub Actions atualizados). Sem mudança de comportamento AdvPL/TLPP — validado com `go build` / `go vet -unsafepr=false` / `go test ./...` limpos e cross-compile linux/windows/darwin do `advplc`.
- **Reorganização de documentação**: arquivos da raiz movidos pra `docs/` (exceto `README.md` / `.pdf` e `CHANGELOG.md` / `.pdf`), removidos 4 docs redundantes/desatualizados (`TDN_FUNCTIONS.md`, `TDN_FRAMEWORK_FUNCTIONS.md`, `TECNICO.md`, `FEATURE_MATRIX.md`), `LIMITS.md` traduzido integralmente pra `pt_BR`.
- **Template de PDF trocado de `clean` pra `jmpm`** (identidade JMPM Tecnologia) nos 6 PDFs de documentação versionados.

Adicionado

- `ROADMAP.md` : dívidas técnicas levantadas (gap `FWMBrowse` sem `SetReadOnly()`, falta de função nativa UTF-8-aware, bug conhecido de `JsonObject.ToJson()` com objeto aninhado, suporte a Windows Service) e levantamento Node.js/Bun/Deno pro toolchain do `web/`.
- Skills `docs-pdf-regen` (global) e `advpp-release` (deste repo) — formalizam os processos de regeneração de PDF e de release.

[3.0.4] — 2026-08-23

Adicionado — auditoria de 27 documentos TDN + gRPC embarcado + Smart Link real

Auditoria linha-a-linha de 27 PDFs de referência TDN (diretivas de preprocessador, criptografia, classes "Não Visual"/"TLPP - Classes úteis", família RPO) contra o runtime, seguida de implementação de todos os gaps reais encontrados e de dois pedidos de acompanhamento (gRPC de verdade, e o Smart Link real descoberto ao auditar código-fonte Protheus 12.1.2510 real):

- **Argon2id** (`pkg/vm/argon2_native.go`): RFC9106 via `golang.org/x/crypto/argon2`.
- **tPBKDF2** (`pkg/vm/pbkdf2_native.go`): classe OOP completa (SHA1-SHA3-512) via `golang.org/x/crypto/pbkdf2`.
- **tHashMap** classe **OOP** (`pkg/vm/matrizhashmap_native.go`): `New/Set/Get/Del/List/Clean/nStatus`, interopera com a API funcional histórica (`HMNew` / `HMSet` / ...).
- **tJsonParser** (`pkg/vm/jsonparser_native.go`): `New/Json_Hash/Json_Parser/json_ok`, parsing JSON real.
- **tUnicode** (`pkg/vm/tunicode_native.go`): `Normalize` (NFC/NFD/NFKC/NFKD via `golang.org/x/text/unicode/norm`) e `ConvertEncoding`.
- **TFtpClient** (`pkg/vm/ftpclient_native.go`): cliente FTP real (RFC 959, modo passivo) — testado ponta a ponta contra um servidor FTP real (upload/download com round-trip verificado).
- **Resource2File** / **GetPatchFile** / **SRCheckSourceSignature** (`pkg/vm/rpo_native.go`): completam a família de funções de manipulação de RPO.
- **2 bugs reais corrigidos no motor** `#command` / `#xcommand` /

`#translate` / `#xtranslate` (`pkg/preprocessor/commands.go`): wild match marker `<*>` e extended-expression match marker `<(x)>` quebrados (nome do marcador nunca era casado); dumb stringify `#<x>` e smart stringify `<(x)>` no resultado nunca eram reconhecidos. Sem regressão na suíte existente.

- **tGrpc** (`pkg/vm/tgrpc_native.go`): cliente gRPC real via `google.golang.org/grpc` — conexão HTTP/2 real, descoberta de serviço/método via **gRPC Server Reflection** em runtime (sem `.proto` local) e invocação dinâmica (`dynamicpb`).
- **GRPCServer** (`pkg/vm/grpcserver_native.go`): o AdvPP agora também é **servidor** gRPC real — `AddMethod` / `Serve` expõem `User Function` como RPCs unárias com Server Reflection habilitada, simétrico ao `WSRestServer` já existente. Testado ponta a ponta com um cliente Go genuíno que descobre e invoca via reflection, sem stub gerado em tempo de compilação.
- **FwTotvsLinkClient** (`pkg/vm/totvslinkclient_native.go`): cliente HTTP real (`net/http`) para o Smart Link — descoberto ao auditar código-fonte real do Protheus 12.1.2510 (TechFin, RH Insights), que usa esta classe em vez de `tGrpc` diretamente (Smart Link é HTTP+fila, não gRPC). OAuth2 `client_credentials` real; endpoint/credenciais/tenant configuráveis via `ADVPP_SMARTLINK_*`.

Todas as novas dependências (`google.golang.org/grpc` v1.71.0, `google.golang.org/protobuf` v1.36.4, `golang.org/x/crypto`, `golang.org/x/text` — as duas últimas já presentes no `go.mod`) compilam e passam nos testes em Linux, Windows e macOS (`go build` / `go vet` cross-compile limpos; `go test ./...` 20/20 pacotes verdes). Detalhe técnico completo de cada item, incluindo limitações reais que permanecem (nomes de método/campo do Smart Link real não confirmados sem o `.proto` /registry proprietário da TOTVS), em `docs/tdn-known-limitations.md`.

[3.0.3] — 2026-08-15

Corrigido — botão "Fechar" duplicado no modal de menu (`FWMenuSelect`) da WebUI

O modal do menu (`web/src/app/app.ts`) definia apenas `p-secondary-action`, deixando `p-primary-action` vazio. O `po-modal` (PO-UI) sempre renderiza um botão de ação primária no rodapé — e, quando nenhuma é informada, gera um próprio com o label padrão "Fechar" — então o resultado eram dois botões "Fechar" lado a lado. Corrigido movendo `menuCloseAction` de `p-secondary-action` para `p-primary-action`, único slot de ação efetivamente usado nesse modal.

Reportado por **Wilson Kraft**, que reproduziu o bug tanto num app de exemplo próprio quanto rodando o GesCon localmente, e apontou a pista certa (`p-primary-action` vazio) direto no `main-*.js` compilado. Valeu, Wilson!

Corrigido — "Sair"/"Exit" duplicado com "Quit" no menu das GUIs desktop (advpp-ide, adveditor)

Mesma família de bug do item acima, só que no lado desktop: quando o `fyne.NewMainMenu` do IDE (`cmd/advpp-ide/main.go`) e do AdvEditor (`cmd/adveditor/main.go`) é montado, o driver desktop do Fyne (`internal/driver/glfw/menu.go`) sempre garante uma opção de saída no primeiro menu — se o último item do menu não tiver o label exato `"Quit"`, o próprio Fyne injeta um item `"Quit"` extra ao lado do que já existia.

- **advpp-ide**: o último item do menu "File" era `"Exit"` → aparecia `"Exit"` e `"Quit"` juntos. Renomeado para `"Quit"`.
- **adveditor**: o último item do menu "Arquivo" era `"Sair"` → aparecia

"Sair" e um "Quit" em inglês, misturando idioma. Mantido "Sair" (consistente com o resto da UI em português) e marcado `IsQuit: true` explicitamente no item, em vez de depender do casamento por string.

Achado ao investigar o mesmo padrão do fix anterior — Peder lembrava de ter visto essa duplicação visualmente numa das GUIs desktop antes.

[3.0.2] — 2026-08-13

Corrigido — DynCall (`tRunDll`) inacessível a partir de AdvPL/TLPP compilado

`TRunDll` (lançado em v3.0.1) nunca foi registrado em `builtinClasses` (`pkg/compiler/codegen.go`) — só existia no `switch` de instanciação do VM, alcançável apenas por chamada Go direta em teste, nunca por `TRunDll():New(...)` escrito em AdvPL/TLPP real de verdade. Corrigido: a classe agora compila e roda a partir de código-fonte AdvPL/TLPP.

Corrigido — `StrCpy` / `MemCpy` do DynCall eram funcionalmente inertes

Os dois métodos documentavam (seguindo a TDN) retorno lógico com o conteúdo lido devolvido por referência num parâmetro de saída (`cRet`) — mas este VM não escreve `@var`, então `cRet` nunca era populado e não havia NENHUMA forma de ler de volta um buffer escrito por uma DLL. Mudança deliberada de contrato (`oDll:StrCpy(oPointer, nMaxSize)` / `oDll:MemCpy(oPointer, nBytes)`) agora **retornam a `String` lida diretamente**, em vez do `Logical` inútil — sem uso real anterior a quebrar, já que a feature era inacessível (ver item acima).

Adicionado — wrapper de compatibilidade `ExecInDLLOpen` (TDN legado)

`examples/dyncall/execindll_compat.prw`: implementa a API clássica TDN de carga de DLL do SmartClient (`ExecInDLLOpen` / `ExecInDLLRun` / `ExeDLLRun2` / `ExeDLLRun3` / `ExecInDLLClose`) sobre `tRunDll`, permitindo rodar scripts legados que dependiam dessa API sem precisar de SmartClient (funciona em AppServer, job ou standalone). Testado ponta a ponta contra uma DLL C real (`cmd/advplc/execindll_compat_test.go`).

Sanitização do repositório

Removidos do controle de versão artefatos que vazaram para o histórico: um PDF sem relação com o projeto (`freenet-whitepaper.pdf`), três binários de exemplo compilados de ~38 MB cada (`examples/chat/chat-peer-*`), um banco SQLite de desenvolvimento local e logs de fuzzing avulsos. `.gitignore` atualizado para não deixá-los voltar.

[3.0.1] — 2026-08-12

Adicionado — DynCall (`tRunDll`): chamada dinâmica de DLL/SO

Implementação real do recurso TLPP DynCall: `New` / `Free` (`dlopen/dlclose` real), `CallFunction` / `CallMethod` (ABI real via `reflect.FuncOf` + `purego.RegisterFunc`, cobrindo toda a legenda de tipos da TDN — `V,B,C,c,S,s,I,i,L,l,G,g,F,D,P,A,T` —, `CallMethod` com resolução de símbolo mangled Itanium C++ real, incluindo construtores), `GetVar` / `SetVar` (leitura/escrita real de variável global exportada pela DLL), `NewPointer` / `NewObj` / `FreeObj` / `StrLen` / `StrCpy` / `MemCpy` (handles opacos, incluindo alocação Go real fixada com `runtime.Pinner` para `NewObj(nBytes)`), `GetTimeout` / `SetTimeout` / `GetLastError` / `GetErrorMsg`. Implementado sem CGO via `github.com/ebitengine/purego`, com resolução de símbolo por SO (`purego.Dlopen` / `Dlsym` / `Dlclose` em Unix, `golang.org/x/sys/windows.LoadLibrary` / `GetProcAddress` /

`FreeLibrary` em Windows, já que purego não implementa aquelas três em Windows). Testado ponta a ponta contra bibliotecas C e C++ reais compiladas por `gcc` / `g++` em tempo de teste, não simulado. Limitações documentadas em `docs/tdn-known-limitations.md`: retorno lógico (não o valor computado, seguindo o mesmo padrão já usado em outras categorias deste compilador para `@var`), mangling só Itanium (sem MSVC), e passagem de parâmetro por referência via `@` não suportada (o compilador AdvPP descarta `@` em qualquer posição de chamada — limitação pré-existente, não nova).

Corrigido — CI

- `go vet` passa a rodar com `-unsafePtr=false` (`Makefile`, `.github/workflows/test.yml`): o único uso de `unsafe.Pointer` do projeto (FFI real do DynCall, endereços vindos de fora do heap Go) aciona o analisador `unsafePtr` por design — os demais analisadores do vet continuam ativos.

[3.0.0] — 2026-08-11

Adicionado — 41 novas natives AdvPL da documentação TDN

Cruzamento do mirror `~/tdn-advpl-mirror` (Functions) contra o runtime: a partir desta versão, 729 natives registradas, cobrindo Banco de Dados, Ambiente e Segurança/Criptografia. A lista completa de pendências restantes (168 funções TDN ainda sem native) está em `docs/tdn-pendencias.md`.

Banco de Dados — tabela (14): `DBChangeAlias`, `DBCclearAllFilter`, `DBCclearIndex`, `DBCcloseAll`, `DBCommitAll`, `DBcreate`, `DBfieldInfo`, `DBfilter`, `DBfilterCB`, `DBgetActFld`, `DBgoTo`, `DBinInsert`, `DBinfo`, `GetDBExtension`.

Banco de Dados — campo/estado (8): `Field`, `FieldBlock`, `FieldWBlock`, `Found`, `Header`, `LastRec`, `NetErr`, `RecSize`.

Banco de Dados — índice/RDD/lock (15): `IndexKey`, `IndexOrd`, `OrdBagName`, `Ordcreate`, `OrdDescend`, `OrdKey`, `OrdListAdd`, `OrdName`, `OrdNumber`, `OrdSetFocus`, `RDDName`, `RDDSetDefault`, `RealRDD`, `FLock`, `RLock`.

Ambiente — funções genéricas (3): `CmpBuildStr`, `GetBuild`, `GetEndPoint`.

Segurança/Criptografia (1): `SMIMESign`.

Notas de comportamento (validadas na revisão de código)

- `DBcreate` valida nomes de tabela/campo contra o identificador AdvPL antes de interpolar no `CREATE TABLE` (vetor de injeção DDL fechado).
- `OrdKey` aceita argumento numérico como posição do índice (1-based), conforme a especificação TDN.
- `RDDSetDefault` retorna a RDD **anterior** quando a padrão é alterada (contrato TDN); com alias vazio, o driver é só anunciado.
- `SMIMESign` assina com certificado X.509 + chave RSA reais (path/PEM) usando RSA/SHA-256 PKCS#1 v1.5 e devolve a assinatura em Base64 no formato pseudo-PKCS7 — implementar o envelope CMS/PKCS#7 real ficou fora do escopo desta versão (documentado no report da Task 34).
- Motor SQLite: `DBfilter` / `DBCclearAllFilter` mantêm o filtro como estado do VM (não há filtro físico) e `DBfieldInfo` aproxima tipos/tamanhos (TEXT/INTEGER); divergências documentadas nos reports das Tasks 33a–33c.

[2.0.22] — 2026-08-09

Adicionado

- `RpcSetEnv(cFilial)` / `FWxFilial(cAlias)` — estado de sessão multi-filial (convenção Protheus: 6 caracteres, GG+UU+FF). `FWxFilial` devolve a filial ativa truncada conforme o nível de compartilhamento configurado por tabela em `X2_FILIAL_COMPART` (default: nível 6, exclusiva — falha segura sem configuração).
- `FWMBrowse` filtra e estampa `FILIAL` automaticamente quando a tabela física tem essa coluna — mesmo mecanismo que já existe para `DELETE`. Motivado pelo multi-condomínio do GesCon.
 - A guarda de escrita cobre Incluir, Alterar e Excluir — não só o Listar (`SELECT`). Alterar reconfere `FILIAL = ?` no `WHERE` do `UPDATE` (não confia apenas no recno já ter passado pelo filtro do Listar) e Excluir faz o mesmo tanto no soft-delete (`DELETE = '*'`) quanto no hard-delete. Um recno forjado/obsoleto de outra filial é recusado silenciosamente (0 linhas afetadas, sem erro) em vez de alterar/excluir a linha errada.
 - `FILIAL` nunca é um campo editável pelo cliente — é gerido pelo sistema, exatamente como `DELETE`. Não aparece na lista de colunas do browse (nem via fallback físico, nem via SX3), então um valor forjado enviado pelo cliente em `Data["FILIAL"]` é ignorado; a linha é sempre estampada/mantida na filial ativa resolvida no servidor.
 - Reconhece apenas uma coluna chamada literalmente `FILIAL` — variantes prefixadas (ex.: `A1_FILIAL`) não são detectadas. É um limite de escopo deliberado desta primeira versão, não um bug.
 - Contrato DDL de `X2_FILIAL_COMPART`, exigido por quem for popular essa tabela (ex.: o instalador do GesCon):

```
CREATE TABLE X2_FILIAL_COMPART (TABELA TEXT PRIMARY KEY, NIVEL INTEGER NOT NULL)
```

Sem uma linha para a tabela consultada, `resolveFilial` usa o default fail-closed (nível 6, exclusiva).
- `RpcSetEnv(cFilial)` valida o tamanho do argumento — deve ter exatamente 6 caracteres (convenção Protheus GG+UU+FF). Um valor de outro tamanho é recusado (`.F.`, filial ativa da sessão permanece a anterior) em vez de ser silenciosamente truncado/espçado por `truncarFilial`, o que colapsaria dois códigos de filial pretendidos distintos no mesmo valor.

[2.0.21] — 2026-08-07

Adicionado

- `FWJOBSTART(cFuncName, params...) -> cJobId` — job assíncrono com retorno de valor, complementar ao `StartJob` / `STARTJOB` existente (que continua fire-and-forget, sem alteração). Dispara a função pelo nome (case-insensitive, aceita `Static Function`) numa goroutine com VM isolado, respeitando o mesmo limite `MaxConcurrentJobs` já usado por `StartJob`. Motivado pelo agente `orchestrator` do `shortcoder`.
- `FWJOBDONE(cJobId) -> lDone` — checagem não-destrutiva de conclusão, separada da leitura do resultado. Resolve uma ambiguidade do design inicial: `FWJOB POLL` usa `Nil` tanto para "pendente" quanto para "job terminou e retornou Nil", com leitura destrutiva — sem `FWJOB DONE`, uma função-alvo que legitimamente retornasse Nil travaria o polling para sempre. `FWJOB START` agora grava um placeholder `done:false` antes de disparar a goroutine, para que a entrada exista desde o início.
- `FWJOB POLL(cJobId) -> uResultado` — `Nil` enquanto pendente; valor

(tipicamente Character) assim que pronto, com leitura única (a entrada é removida do armazenamento interno ao ser lida).

[2.0.20] — 2026-08-06

Adicionado

- **Line editor com histórico no `ConIn`** (`pkg/vm/lineeditor.go`) — motivado pelo uso real no projeto `shortcoder`. Em terminal interativo de verdade (`term.IsTerminal`), `ConIn` entra em raw mode e roda um mini line-editor: setas `t/↓` navegam as últimas 20 linhas digitadas (ignora repetição consecutiva, como `HISTCONTROL=ignoredups` do bash), `←/→/Home/End/Ctrl+A/` `Ctrl+E` movem o cursor, `Backspace/Delete` editam, `Ctrl+U/Ctrl+K` apagam até o início/fim da linha, `Ctrl+L` limpa a tela, `Ctrl+C` cancela a linha atual. Em stdin não-interativo (pipe/redirect) o comportamento continua idêntico ao anterior — sem mudança de API para scripts existentes.
- **`FWHTTPHEADER(cName, cValue)` / `FWHTTPCLEARHEADERS()`** — headers customizados (ex.: `Authorization: Bearer ...`) aplicados a toda requisição `FWHTTPGET/POST/PUT/PATCH/DELETE` seguinte, até serem limpos.
- **`FWHTTPTIMEOUT(nSeconds)`** — timeout configurável por requisição HTTP (default seguia fixo em 30s).

[2.0.19] — 2026-08-02

Adicionado

- **Primitivas de TUI de terminal** (`pkg/vm/ui_render.go`), estilo `opencode/Claude Code` — um programa `AdvPL/TLPP` compilado com `advplc build` agora pode compor a própria interface de terminal em vez de só usar os diálogos prontos (`MSDIALOG` / `FWGetText` / `FWMenuSelect`):
 - `UiBox(cTitle, cBody, cColor, [nWidth])` — caixa com borda arredondada (lipgloss)
 - `UiStreamBox(...)` / `UiStreamReset()` — mesma caixa, mas auto-redesenha por cima da anterior a cada chamada (streaming de LLM token a token, efeito de "cartão crescendo ao vivo")
 - `UiMarkdown(cMarkdown, [nWidth])` — markdown renderizado em ANSI via `glamour` (negrito, itálico, listas, código, títulos)
 - `UiAltScreenEnter()` / `UiAltScreenExit()` — tela alternativa do terminal (buffer do `vim/less/htop`), com handler de `Ctrl+C` que restaura a tela normal antes de encerrar
 - `UiTermWidth([nDefault])` — largura real do terminal, com fallback quando `stdout` não é TTY
- **`ConOutRaw(cText)`** — escreve sem `newline`, para texto em streaming na mesma linha do terminal.
- **`ProcRun(cPath, aArgs, bOnStdoutLine, [bOnStderrLine])`** — executa um processo filho (sem shell) com callback síncrono por linha de `stdout/stderr` — consumo de NDJSON em streaming (ex.: CLI de um LLM) direto de dentro do `AdvPL`.
- Nova dependência: `github.com/charmbracelet/glamour` (+ `go-colorable`) para paridade de cores ANSI no Windows, aplicado a toda a saída de console: `CONOUT` / `CONOUTW` / `CONIN` / `CONOUTRAW` e as natives de TUI acima).

Corrigido

- **`JsonObject:FromJson()`** era um stub que sempre devolvia `Nil` sem

parsear nada. Agora é um parser real (`encoding/json` da stdlib): objetos aninhados viram `JsonObject` (acesso via `oJ["chave"]`), arrays viram `Array` 1-based, `null` vira Nil. Em JSON inválido ou top-level não-objeto, devolve `.F.` sem mutar o objeto e sem lançar erro.

- **`ConIn()` não distinguia EOF real de linha vazia.** Se o stdin fechasse de verdade (Ctrl+D, pipe esgotado), `ConIn()` devolvia `""` — igual ao caso normal de "usuário só apertou Enter". Agora devolve Nil no EOF real (checar com `IsNil()`), permitindo que um REPL saia do loop em vez de reimprimir o prompt pra sempre.

[2.0.18] — 2026-08-02

Bug relatado por **Wilson Kraft**, que testou o renderer web do AdvPP com um programa próprio de cadastro de contatos (`FWMenuSelect` + `FWMBrowse`) e identificou o comportamento de reinício descrito abaixo — obrigado pelo relato detalhado, incluindo o log técnico e o código de reprodução, que tornaram possível isolar a causa raiz de primeira.

Corrigido

- **`advplc serve` : reconexão do EventSource reiniciava o programa do zero quando havia um diálogo pendente.** O browser reconecta sozinho no `/events` sempre que a conexão SSE cai (proxy, aba suspensa, blip de rede) — sem recarregar a página, reusando o mesmo id de sessão. O handler tratava toda reconexão como sessão nova: recriava o estado e reexecutava o programa inteiro, enquanto a goroutine anterior ficava presa pra sempre esperando uma resposta que nunca chegaria. Sintoma visível: um `FWMenuSelect` (ou qualquer outro diálogo) parado esperando o usuário, numa 2ª tela do fluxo, e do nada o log técnico "reiniciava" sozinho — nenhuma linha de código rodando entre a ação do usuário e o reset aparente. Agora uma reconexão com id de sessão já conhecido e ainda em execução retoma o mesmo stream em vez de recriar tudo; a sessão só é descartada quando o programa realmente termina.
- **`FWMenuSelect` no renderer web não tinha como fechar sem escolher uma opção.** O popup não tinha botão de fechar nem reagia a ESC/clique fora — a única saída documentada (fechar sem escolher devolve `0` pro código chamador) não existia na prática, e qualquer tentativa de fechar deixava a execução bloqueada indefinidamente esperando uma resposta que nunca seria enviada. Adicionado o botão "Fechar", que agora devolve `0` corretamente — mesmo contrato do `FWMenuSelect` real.

[vscode-2.0.17] — 2026-08-01

Corrigido

- **`Ctrl+F9` (Build standalone executable) não passava `--gui` .** O binário resultante ficava sujeito à heurística de auto-detecção console-vs-GUI em runtime — no Windows, isso significa subsistema console (uma janela de console pode aparecer atrás da janela Fyne) e, pior, um duplo-clique no `.exe` faz o stdin parecer um TTY, levando o binário para o caminho de console — que não tem suporte a `MSDIALOG` nenhum. Programas com `MSDIALOG` buildados pelo atalho simplesmente não renderizavam o diálogo no Windows.

Agora `Ctrl+F9` passa `--gui` por padrão (nova config `advpl.buildGui`, default `true`) — sem efeito em programas sem UI nenhuma (que rodam em console de qualquer forma), mas garante `MSDIALOG` funcional e binário

Windows linkado no subsistema GUI (`-H=windowsgui`). Desligável via configuração para quem prefere o fallback de terminal interativo ao rodar o binário de um terminal de verdade.

[2.0.14] — 2026-08-01

Corrigido

- **O Mesa3D instalado ao lado dos executáveis impedia as GUIs de abrirem.**

`opengl32.dll` é import **estático** dos binários Fyne: o Windows o carrega na criação do processo, antes de qualquer código nosso. Numa VM QEMU/QXL o Mesa não inicializa — provavelmente instruções de CPU que o `libgallium_wgl.dll` usa e a vCPU não tem — e o processo morre no carregador: sem janela, sem saída, sem log de aplicação. `advpp-ide.exe` saía com `0x80070057` e nada mais.

Instalar o Mesa por padrão trocava um erro legível — *"WGL: The driver does not appear to support OpenGL"* — por silêncio total. Agora ele vai para `{app}\mesa\`, fora do caminho de busca de DLL, e só entra em jogo por `Ativar-renderizacao-por-software.bat`. Há também o script que desfaz, sem o qual sair da situação exigiria saber quais arquivos apagar.

A heurística que adivinhava, pelo registro, se a máquina tinha driver OpenGL foi removida junto com a caixa que ela marcava. Ela errou nessa VM, e o preço do erro não era "roda mais devagar", era "não abre e não diz nada".

[2.0.13] — 2026-08-01

Corrigido

- **O instalador abortava ao extrair o toolchain:** *"O formato do arquivo compactado não é compatível"*, para o `go1.24.2.windows-amd64.zip` — um zip deflate perfeitamente válido, com SHA-256 conferido pelo próprio download do Inno e que o 7-Zip lê sem reclamar. O extrator do `ArchiveExtraction` simplesmente não dá conta dele.

Trocado pelo `tar.exe` do próprio Windows (bsdtar, presente desde o Windows 10 1803), que extrai zip nativamente. Se ele não existir, o instalador avisa e segue sem o toolchain em vez de baixar 350 MB para nada.

[2.0.12] — 2026-08-01

Adicionado

- **`advplc build` deixa de exigir Go no PATH.** O comando embute o bytecode num stub Go e chama `go build`, o que precisa do toolchain Go e — porque o stub importa o Fyne — de um compilador C, já que CGO é obrigatório (sem ele o build morre em `go-gl/gl: build constraints exclude all Go files`).

O `advplc` agora procura uma pasta `toolchain/` ao lado do próprio executável: `toolchain/go/bin/go` e um `gcc` em qualquer subpasta com `bin/`. Nada precisa entrar no PATH. `ADVPP_GO` continua tendo precedência, e sem a pasta o comportamento é exatamente o de antes — `go` e `gcc` vêm do PATH. `run`, `check` e `serve` não passam por aqui.

O instalador do Windows ganhou a opção *Baixar o toolchain de compilação*, que traz Go 1.24.2 (com SHA-256 conferido) e o mingw-w64 durante a instalação, ~350 MB. Ela vem marcada quando não há Go na máquina, e falha de rede não derruba a instalação: avisa, desmarca e segue — o AdvPP é útil sem

ela. Embutir os dois no pacote levaria o instalador de 43 MB para ~760 MB por causa de um subcomando.

O instalador passa também a levar os fontes do módulo Go (`{app}\advpp-src`, ~7 MB), consultados por `findModuleRoot` como último recurso — `ADVPP_SRC` e o checkout do desenvolvedor seguem com precedência.

Corrigido

- O ambiente do `go build` era montado com `append` em `os.Environ()`, o que deixava a mesma chave duas vezes quando ela já existia; qual delas vale fica por conta da plataforma. Agora substitui.

[2.0.11] — 2026-08-01

Corrigido

- `adveditor` e `advpp-ide` morriam quando abertos por atalho para dentro de `Program Files`. Mesma causa raiz que a 2.0.10 corrigiu no stub, um nível acima: o passo 4 do `ResolveDatabasePath` devolve `./advpp.db`, e o diretório de trabalho de um atalho é a pasta de instalação. O SQLite não cria nada ali, quem chamou recebe um engine nulo, e o erro só aparece adiante — como falha de query, ou como janela que fecha sem mensagem.

Agora o passo 4 verifica se dá para escrever: banco já existente no diretório manda (gravável ou não — trocar o caminho por baixo de quem já usa substituiria o banco em uso por um vazio), depois o diretório atual se gravável, depois a pasta de dados do usuário. Os passos 1 a 3 não mudaram. A verificação é uma escrita de teste, não um `os.Stat`: no Windows a permissão efetiva depende da ACL, e `Program Files` parece um diretório comum até a hora de gravar.

Confirmado em campo: o mesmo executável que não abria com duplo clique em `Program Files` subia normalmente quando lançado de um `.bat` numa pasta do usuário — a única diferença era o diretório de trabalho.

- **App standalone deixa de guardar o banco no diretório de trabalho.** Duas consequências, ambas relatadas em campo: o mesmo app lançado da Área de Trabalho e de Documentos usava dois bancos, partindo o dado do usuário sem nenhum aviso; e quando por acaso caía na mesma pasta de uma ferramenta AdvPP, os dois disputavam o mesmo arquivo — era o que impedia GesCon e `advpp-ide` de ficarem abertos ao mesmo tempo. Agora `ResolveStandaloneDatabasePath` devolve sempre a pasta de dados do usuário, numa subpasta com o nome do app: `ADVPP_DB` se definida, e um caminho só em qualquer outro caso.

As ferramentas (`advplc`, `adveditor`, `advpp-ide`) mantêm o `./advpp.db` do diretório de projeto — é o que faz o editor enxergar o banco que o compilador acabou de usar. Quem quiser banco por diretório num app standalone define `ADVPP_DB`.

[2.0.10] — 2026-08-01

Corrigido

- **Erro na partida de um executável standalone sumia da tela.** O stub rodava o VM numa goroutine e chamava `w.Close()` assim que ela terminasse. Erro precoce — banco que não abre, função que não existe — acontece em milissegundos, então o `Close` corria junto com o `ShowAndRun`: a janela

fechava antes de pintar e o programa desaparecia sem mensagem. No subsistema GUI do Windows não há stderr visível, então o usuário só via "o aplicativo não abre". Agora a janela fica aberta com o erro e o caminho do banco em uso.

- **Banco de um app distribuído nascia no diretório de trabalho.** O stub usava `ResolveDatabasePath`, que cai em `./advpp.db` — premissa válida para a CLI, que roda num diretório de projeto, e errada para um app lançado com duplo clique dentro de `Program Files`. O SQLite não abria nada, a fábrica devolvia `nil`, e a primeira chamada de `TCSqlExec` derrubava o programa. Novo `ResolveStandaloneDatabasePath`: `ADVPP_DB` > banco já existente no diretório atual > diretório atual, se gravável > pasta de dados do usuário. O segundo passo preserva quem já usa o app — mudar o caminho por baixo trocava o banco em uso por um vazio.

Conhecido

- `Try/Catch` não captura erro levantado por função nativa: o erro sai de `callNative` como erro Go e passa por cima do frame do `try`, abortando o programa. Um `.prw` não tem como se defender de `TCSqlExec` sem banco. Tornar erro de nativa capturável é mudança de semântica do VM e fica para uma versão própria.

[2.0.9] — 2026-08-01

Adicionado

- **Instalador do Windows** (`AdvPP-Setup-<versão>.exe`, Inno Setup). O zip solto não entrega o que faz uma toolchain ser usável: o instalador põe `advplc` no PATH da máquina, cria atalhos para a IDE e o AdvEditor, e desinstala. Traz também o Mesa3D como opção marcável — `advpp-ide` e `adveditor` são Fyne e morrem com *"WGL: The driver does not appear to support OpenGL"* em máquina sem driver de vídeo ou em VM; a opção vem pré-marcada quando nenhum driver OpenGL está registrado. Fora do escopo de propósito: `advplc build` compila o bytecode dentro de um stub Go, então continua exigindo o Go e o repositório do AdvPP (`ADVPP_SRC`) na máquina. Embutir o Go trocava um download de 25 MB por um de 200 MB por causa de um subcomando. O instalador avisa quando não acha o Go; `advplc run|check|serve` funcionam sem ele.

[2.0.8] — 2026-08-01

Corrigido

- **Cabeçalho de coluna do browse invadia a coluna vizinha.** A largura vinha só do `X3_TAMANHO`, que descreve o *dado*, não o título. `DES_COMPET` tem 7 caracteres de dado e título "Competência" com 11, então o cabeçalho saía colado no seguinte — na grade de Despesas lia-se `CompetênciaData Lançamento`. Agora a largura é o maior entre dado e título. A contagem usa runes, não bytes: caractere acentuado ocupa mais de um byte em UTF-8 e inflaria a coluna sem necessidade.

[2.0.7] — 2026-08-01

Adicionado

- `advplc build --gui`: marca o programa como app desktop. O executável

sempre abre a janela Fyne em vez de cair na UI de terminal, e no Windows é linkado no subsistema GUI (`-ldflags -H=windowsgui`).

Corrigido

- **Executável Windows caía na UI de terminal.** `advplc build` produzia um binário de console. No Windows o duplo-clique aloca um console, então `IsTerminal(os.Stdin)` respondia verdadeiro, o stub escolhia o `TerminalUIProvider` — que não implementa diálogos — e todo `MSDIALOG` morria com "requer o modo web". Não havia contorno prático: a variável `ADVPP_FORCE_GUI` exige um shell, e quem clica no `.exe` não tem um. Compile com `--gui`.
- **`advplc build ... -o` era ignorado depois de qualquer flag.** O caminho de saída só era lido quando `-o` caía exatamente em `os.Args[3]`; em `build app.prw --gui -o App` o binário saía calado em `./output`. Agora `-o` é procurado em qualquer posição.

[2.0.6] — 2026-08-01

A partir desta versão o compilador e a extensão do VS Code compartilham o mesmo número: a extensão vinha de 2.0.5 enquanto o último tag do compilador era v2.0.3, e a divergência dificultava saber qual `advplc` estava embutido.

Correção: campos de `GET` do `MSDIALOG` saíam com largura fixa mínima no Fyne

Todo campo de entrada de um `MSDIALOG` era renderizado com a largura mínima do `widget.Entry` dentro de um `HBox` — uma caixa de dois ou três caracteres, independente de o campo esperar uma data, um código de conta ou um histórico inteiro. Uma data `20260731` aparecia cortada em `2026`.

O `SIZE` declarado no fonte era simplesmente descartado: o parser já o entregava como par de argumentos, mas `AT_GET` não o lia e o provider Fyne criava o `Entry` sem tamanho algum.

Corrigido:

- `pkg/vm/dialog.go`: `AT_GET` captura a largura de `SIZE <w>,<h>` (novo helper `dialogClauseFloat`) e a envia em `dlgControl.Width`
- `pkg/ui/msdialog.go`: `entryWidth` decide a largura na ordem de quem sabe mais sobre o dado — `SIZE` declarado, senão o tamanho da `PICTURE`, senão o valor atual — com piso e teto para não degenerar. Como `widget.Entry` não expõe `MinSize` gravável, o tamanho vem de um `GridWrapLayout`.

Diálogos que não declaram `SIZE` também melhoram: o piso é bem maior que o `MinSize` padrão do widget.

Achado no GesCon, ao usar os formulários de lançamento contábil, abertura de exercício e configuração de boleto.

[2.0.3] — 2026-07-29

Correção crítica: console real no `advplc build` — login e `FWMBrowse` não funcionavam

Até esta versão, um executável standalone (`advplc build`) rodando em modo headless compilava e "executava" sem erro mesmo quando a interação real nunca acontecia: `FWGetText` / `FWMenuSelect` sem nenhum `UIProvider` anexado devolviam o valor default silenciosamente (uma tela de login "passava direto" sem pedir usuário/senha, sem imprimir nada), e `FWMBrowse` sempre falhava com "requer o modo web (advplc serve)" mesmo fora do modo web. Nenhum teste automatizado cobria esse caminho — só existia um smoke test 100%-console, sem nenhum prompt. Descoberto ao tentar usar um app real (e-Gov) interativamente pela primeira vez.

Corrigido:

- Console interativo de verdade via `TerminalUIProvider` (`pkg/ui/terminal.go`, `huh` + `lipgloss`): formulários com borda, seleção por seta, tema escuro fixo para não depender de uma query OSC 11 ao terminal (que pode travar em ambientes que não respondem)
- `FWMBrowse` agora renderiza no console — listagem em tabela (largura adaptada ao terminal, com aviso de campos ocultos), Novo/Editar/Excluir via formulário multi-campo
- Detecção console-vs-GUI corrigida: a heurística antiga varria `bc.Classes` (só classes *declaradas*, nunca detectava `FWMBrowse():New()`, o padrão de uso real); agora varre o bytecode de verdade (`OP_CALL_NATIVE` / `OP_NEW_INSTANCE`)
- `ADVPP_FORCE_GUI=1`: força a janela Fyne mesmo rodando de um terminal, por aplicação, sem mudar o padrão (console-se-tem-TTY) dos demais programas AdvPP
- Bug correlato em `FWMBrowse`: o fallback de colunas sem entrada no SX3 expunha `R_E_C_N_O_` / `R_E_C_D_E_L_` (chave interna) como campo editável — um Novo/Editar sobrescrevia o próprio rowid e o INSERT/UPDATE seguinte falhava com "datatype mismatch"

Teste de integração novo: `cmd/advplc/build_standalone_interactive_test.go`

(`TestBuildStandaloneInteractive`) builda o binário standalone, roda sob um pseudo-terminal real e dirige login + `FWMenuSelect` + CRUD completo via `FWMBrowse` exatamente como uma pessoa digitando — é o teste que teria pego esta classe de bug antes de qualquer app real bater nela.

Roda nas três plataformas do release: Linux/macOS via `creack/pty`, Windows via `github.com/UserExistsError/conpty` (ConPty real, Go puro, sem cgo) — mesmo teste, mesma cobertura nos três SOs.

Ver `COMPONENT_STATUS.md` → "Executável Standalone — Console/TUI" para detalhes.

[2.0.1] — 2026-07-28

Cliente HTTP Nativo com Suporte a Certificado PKCS#12

O AdvPP agora possui 8 funções nativas para realizar requisições HTTP com suporte a certificados digitais A1/A3 no formato PKCS#12 (.pfx/.p12).

Funções implementadas:

- `FWHttpGet(cUrl, cCertPath, cCertPass) As Numeric` — requisição GET
- `FWHttpPost(cUrl, cJsonBody, cContentType, cCertPath, cCertPass) As Numeric` — POST
- `FWHttpPut(cUrl, cJsonBody, cContentType, cCertPath, cCertPass) As Numeric` — PUT
- `FWHttpPatch(cUrl, cJsonBody, cContentType, cCertPath, cCertPass) As Numeric` — PATCH
- `FWHttpDelete(cUrl, cCertPath, cCertPass) As Numeric` — DELETE
- `FWHttpBody() As Character` — corpo da última resposta
- `FWHttpStatus() As Numeric` — código HTTP da última resposta
- `FWHttpError() As Character` — mensagem de erro da última requisição

Detalhes técnicos:

- Timeout de 30s por requisição
- TLS com verificação de certificado habilitada (`InsecureSkipVerify=false`)
- Certificados .pfx/.p12 decodificados via `golang.org/x/crypto/pkcs12`
- Implementado em `pkg/vm/httpclient_native.go`
- Testes de integração com servidor HTTP local (`tests/http_native_test.prw`)
- Versão do VSIX advpl-tlpp bumpada para 2.0.1

[1.24.0] — 2026-07-24

`FWGetText` com 3º argumento opcional `bIsPassword` — campo de senha em web e desktop. Motivado pelo GesCon (Plano 2).

`FWGetText` com `bIsPassword` — campo de senha em web e desktop

`FWGetText(cPergunta, cDefault, .T.)` abre um campo de texto oculto (* ou • em web/desktop). Terceiro argumento é opcional: sem ele, comportamento inalterado. No desktop Fyne, o widget do entry troca para `Password=true`; no renderer web (Angular), o input HTML muda para `type="password"` via PO-UI `PoDynamicFormField[type: 'password']`.
Motivada pela necessidade de esconder senhas no GesCon (login/admin cria senha visível na v1). Implementado em: natives.go (3º arg opcional), vm.go (interface `UIProvider.InputText` adiciona `bIsPassword bool`), provider.go (Fyne `PasswordEntry`), server.go (webui transmite pw flag ao frontend Angular), app.ts (`InputSpec.pw → po-dynamic-form password type`).

`FWHash(cTexto) As Character` — hash SHA-256

O AdvPP não tinha nenhuma função de hash (nem MD5, nem SHA, nem o `HB_MD5` real do Protheus) — a única forma de guardar uma senha seria texto puro. `FWHash` usa `crypto/sha256` da stdlib do Go (sem CGO, sem dependência nova), retorna o digest em hexadecimal (64 caracteres). Não existe em Protheus real — capacidade própria do AdvPP. Achado implementando login no GesCon.

Bug real: `MsgInfo` / `MsgAlert` / `MsgStop` não bloqueavam no desktop (Fyne)

No backend desktop (`advplc build` / `advpp-ide`), `MsgInfo` / `MsgAlert` / `MsgStop` disparavam o diálogo e retornavam na hora — a VM seguia em frente sem esperar o usuário sequer ver a mensagem. Num loop de menu (`Do While .T. ... FWMenuSelect(...) ... EndDo`), isso empilhava um diálogo novo por cima do anterior a cada vez que um aviso/erro aparecia antes do próximo `FWMenuSelect` — sensação real de app quebrado ao clicar em qualquer ação que terminasse em aviso. O backend web já bloqueava corretamente (mesmo mecanismo ask/reply do diálogo/browse); só o desktop tinha essa lacuna. Corrigido com o mesmo padrão de canal bloqueante já usado por `MsgYesNo`. Achado reproduzindo o GesCon de ponta a ponta num display real.

Desktop: proporção da janela e ícone de menu

- Janela do standalone build (`advplc build`) ganhou um cabeçalho com o título do app acima do console, e tamanho default mais proporcional (720×480, centralizada) — antes era um console preto vazio atrás de um diálogo pequeno flutuando no meio, sensação de tela desproporcional/quebrada relatada em uso real.
- Heurística de ícone do menu (`FWMenuSelect`) tinha um bug de acentuação: `condom[ii]nio` não casava com "Condôminos" (a forma plural do residente, ô, tem acentuação e posição diferente de "condomínio", a propriedade, í) — caía sempre no ícone genérico. Corrigido em web e desktop com um padrão único cobrindo as duas formas (`cond[oô]m[ií]?n`).

Design: identidade visual própria + menu com ícones (web e desktop)

O renderer web (`advplc serve`) e o executável standalone (`advplc build`) usavam a paleta roxa padrão do PO-UI/Fyne, sem nenhuma identidade própria, e o menu novo (`FWMenuSelect`) era uma pilha de botões cheios sem ícone — funcional, mas cru. Achado construindo o GesCon.

- **Cor de marca própria:** azul-petróleo (`#0f6e68`) nos dois backends — web via `--color-brand-01-*` (cascade automático pros estados de hover/pressed via `--color-action-*`, ver `web/src/styles.css`) e desktop via `pkg/ui/theme.go` (`ui.NewTheme()`), usado por `advpp-ide` e pelo stub standalone).
- **Menu com ícone por item:** heurística de palavra-chave em português (unidade → prédio, condômino → pessoas, despesa → documento, cobrança → recibo, fechamento → histórico, mala → e-mail, sair → logout), espelhada nos dois frontends — web (`web/src/app/app.ts`, `MENU_ICON_RULES`, ícones AnDes) e desktop (`pkg/ui/provider.go`, `menuIconRules`, ícones Fyne). Sem nenhuma informação de ícone vinda do `FWMenuSelect` em si — puramente cosmético no renderer, zero mudança de API.
- **Log técnico escondido por padrão** (web): a saída de `ConOut` bruta ficava sempre visível, ocupando a tela toda — agora fica atrás de um toggle discreto ("Log técnico (N linhas)"), sem sumir de vez.
- **Ícones nos botões do browse desktop** (Novo/Editar/Excluir/Voltar ao menu) e **título de janela real no standalone** (`advplc build gescon.prw -o App` agora abre "Gescon", não "AdvPP" genérico — `BuildStandalone` ganhou um parâmetro `title`, derivado do nome do fonte).

Navegação multi-tela em `advplc serve`: `FWMenuSelect` / `FWGetText`

Até aqui, `advplc serve` só sabia executar UM programa até o fim (ou até um `FWBrowse` ficar aberto) — não havia nenhum jeito de um projeto multi-tela real (ex.: GesCon) oferecer um menu pra navegar entre telas sem reiniciar o processo/trocar de porta. Duas novas capacidades, motivadas por esse caso real:

- `FWMenuSelect(aItens, [cTitulo]) As Numeric` — mostra uma lista de opções e bloqueia até o usuário escolher uma; retorna o índice 1-based escolhido (0 se fechado sem escolher).
- `FWGetText(cPergunta, [cDefault]) As Character` — pede um texto ao usuário e bloqueia até a resposta (ou `cDefault` se cancelado).

Não existem em Protheus real — são capacidades próprias do AdvPP. Sem `UIProvider` (execução headless via `advplc run`, ex. em teste automatizado), ambas retornam o "sem escolha" (`0` / `cDefault`) na hora, nunca bloqueiam esperando input que não vai vir.

Implementado nos dois backends de UI do AdvPP: `advplc serve` (novo evento SSE `menu` / `input`, mesmo protocolo blocking-ask/reply já usado por diálogo e browse — ver `pkg/webui/server.go` — com componente novo no frontend Angular/PO-UI, `web/src/app/app.ts`) e desktop Fyne (`pkg/ui/provider.go`, diálogo customizado com um botão por opção / `EntryDialog`).

[1.22.1] — 2026-07-24

Dois bugs reais de VM/compilador achados construindo o GesCon (sistema de gestão condominial multi-arquivo, primeiro consumidor real de projeto AdvPL com múltiplos `.prw` chamando funções uns dos outros via `#include`). Sem esses dois fixes, `advplc run` / `serve` em qualquer projeto multi-arquivo real corre risco de rodar a função errada ou crashar.

Bug real: comparar variável nunca atribuída a `Nil` derrubava a VM (SIGSEGV)

`Local x` sem inicializador, ou um parâmetro de função não passado numa

chamada com menos argumentos que o declarado, deixava o slot com o zero-value da interface Go (nil de verdade) até a primeira atribuição — em vez do valor `advplrt.Nil` do próprio AdvPP. `x == Nil` (padrão comuníssimo pra parâmetro opcional) chamava `.Equals()` num ponteiro nil e crashava com `SIGSEGV` em `opComparison`. Corrigido: todo frame agora aloca `Locals` via `newLocals()`, pré-preenchido com `advplrt.Nil`. Achado junto com o bug do ponto de entrada abaixo (a função errada sendo chamada sem argumentos expôs o crash).

Bug real: ponto de entrada implícito escolhia a função errada com `#include`

Um arquivo `.prw` sem código de nível superior (só `User Function` s) roda implicitamente a **primeira** função declarada — mas `#include` cola o texto da lib no topo do arquivo, então uma lib auxiliar de UMA função (ex.: `db.prw` com só `GcSqlLit`) virava, sem aviso, o "programa" que `advplc run / serve` executava, no lugar da função de verdade do arquivo que a incluiu. Corrigido: o compilador agora prefere a primeira função declarada **no próprio arquivo raiz** (via novo campo `Preprocessor.RootBoundaryLine / ast.Program.RootBoundaryLine`, marcando onde o conteúdo do arquivo raiz começa no texto já expandido pelos `#include` s), caindo no comportamento antigo (primeira função de todas) só quando o arquivo raiz não declara função própria nenhuma. Preserva os fixtures existentes que dependem da convenção oposta (função de entrada declarada primeiro, callbacks depois — `tests/mcp_test.prw`, `tests/rest_server_test.prw`). Achado durante o desenvolvimento do GesCon (sistema de gestão condominial multi-arquivo).

[1.22.0] — 2026-07-24

`TCSqlExec / TCSqlQuery`, persistência real de `DbAppend / RecLock / FieldPut / MsUnlock`, e classe `TMailMessage` — todas motivadas por uso real (GesCon). Mais um bug real corrigido no caminho: `Recover` sem variável nomeada corrompia o slot local 0 da função.

Bug real: `Recover` sem variável nomeada corrompia o slot local 0

`Begin Sequence ... Recover ... End Sequence` (sem `Using oErr`, forma bem comum de só engolir o erro) escrevia o objeto de erro no **slot local 0** da função — qualquer variável declarada primeiro na função, sem relação nenhuma com o `catch`, tinha seu valor silenciosamente sobrescrito. Causa: `TryCatch.CatchVarIdx` não tinha valor-sentinela pra "sem variável de catch" — o zero-value do Go (`0`) colidia com um índice de slot real. Corrigido inicializando `CatchVarIdx: -1` em `OP_TRY_BEGIN`. Achado testando a classe `TMailMessage` (abaixo) contra esse exato padrão.

`TCSqlExec / TCSqlQuery` — SQL direto pra User Function

A API clássica de work-area (`DbAppend / RecLock / FieldPut / MsUnlock`) não persistia dados de verdade até este release — só `FWMBrowse` gravava (via código Go interno acionado por clique na UI web). Lógica de negócio que precisa gravar programaticamente (loop de inserção, processamento em lote) não tinha caminho nenhum. `TCSqlExec(cQuery)` (INSERT/UPDATE/DELETE) e `TCSqlQuery(cQuery)` (SELECT, devolve array de `JsonObject`) expõem a mesma camada SQL que o `FWMBrowse` já usa internamente. Achado motivado por uso real (GesCon, sistema de gestão condominial construído sobre o AdvPP).

`DbAppend / RecLock / FieldPut / MsUnlock` — persistência real

A API clássica de work-area do AdvPP era stub: `DbAppend()` não criava registro nenhum (`RecCount()` não mudava), `FieldPut` (via `alias->campo := valor`) só mutava memória, `MsUnlock()` não gravava nada, `FieldPos` sempre devolvia `0`. Corrigido: `DbAppend()` insere um registro real (`INSERT`,

valores em branco tipo-apropriados por coluna), `MsUnlock()` grava o registro corrente via `UPDATE`, `FieldPos` resolve a posição física real. Achado confirmado por teste direto durante o planejamento do GesCon.

Classe `TMailMessage` — envio de e-mail real (`net/smtp`)

Nova capacidade de compilador: `TMailMessage():New()` + `SetServer` / `SetAuth` / `SetFrom` / `AddTo` / `SetSubject` / `SetBody` / `Send()` envia e-mail via SMTP real (stdlib `net/smtp`, sem CGO, sem dependência externa). Motivada pela necessidade futura de mala direta do GesCon — capacidade adicionada ao compilador antes do consumidor de verdade (decisão consciente do usuário, adiantando um item que já estava no roadmap).

[1.21.0] — 2026-07-23

Servidor REST executando de verdade (`WSRestServer`) e uma varredura extensa de gaps de linguagem contra os corpora reais Protheus (811R4 + 12.1.2510): 21 bugs de parser/lexer/compiler/VM corrigidos em 4 rodadas, incluindo um bug real de runtime (macro-eval `&expr` derrubava o processo ao executar) e uma regressão própria pega e corrigida na mesma sessão. Amostra fresca de 1800 arquivos reais nunca vistos: 92,7% → 98,7% de aprovação em `advplc check`.

Quarta rodada: "tente ao menos novamente" — 10 gaps + 1 regressão própria

Reanálise dirigida dos fontes-padrão que sobraram nas 3 rodadas anteriores (34 falhas na amostra de 1800), com repro mínimo isolado pra cada um antes de corrigir. Amostra final: 132 → 24 falhas (98,7%) na mesma amostra de 1800 desde o início desta missão de gaps.

- **Regressão real na própria correção de `ident¯o` desta sessão:** o parser aceitava `&` logo após um identificador SEM checar mesma linha — como newlines já foram removidas do stream de tokens antes do parser, um `For var To nOutraVar` (limite do loop é uma variável solta) seguido, na PRÓXIMA linha, de um `&(macro) := valor` de verdade tinha o `&(macro)` colado ao limite do loop (`nOutraVar&(macro)`), e o `:=` real sobrava como token inesperado. Corrigido exigindo mesma linha, igual ao mesmo tipo de guarda já usado alhures neste parser.
- **`Store Header/Cols ... FOR <cond>` seguido de um `For` de loop de verdade na próxima linha:** sem checar mesma linha entre cláusulas, o `For` do loop seguinte era engolido como se fosse mais uma cláusula do `Store`.
- **`Copy <alias> To Memory` sem nome de array (idioma real sem argumento):** o parser sempre exigia um nome depois de `Memory`, engolindo o primeiro identificador do PRÓXIMO statement como se fosse esse nome.
- **`ParamType N Var x As Tipo` seguido de um `Default y := valor` de verdade na próxima linha:** as cláusulas opcionais `OPTIONAL` / `DEFAULT` do `ParamType` não checavam mesma linha, então um comando `Default` real na linha seguinte era engolido como se fosse a cláusula `DEFAULT` do `ParamType` anterior.
- **`alias->¯o.` (nome de campo computado por macro, com o ponto terminador explícito do Clipper):** o ponto não era consumido depois do identificador da macro nesse contexto específico (já era em `¯o.` isolado), sobrando como token inesperado.
- **`REPLACE campo WITH expr ALL FOR <cond>` (cláusulas de escopo do Clipper):** `ALL` / `REST` / `NEXT` / `RECORD` / `FOR` / `WHILE` não eram consumidas depois do(s) par(es) `campo WITH expr`, e um `FOR <cond>` desses vazava pro parser como se fosse um `For` de loop de verdade.
- **`End For` (duas palavras) fechando loop:** só `End` / `EndFor` sozinhos eram reconhecidos; o `For` sobrando depois de um `End` vazava como se fosse

um novo loop.

- `arr[nAux+=1]` (atribuição composta como índice de array): índices de [...] só aceitavam expressão simples, não atribuição (`:=` / `+=` / `...`) — idioma real em codeblocks de iteração paralela.
- Investigado e mantido como falha conhecida (baixo valor, arquivo vendor de ~2400 linhas exigiria bissecção cara pra achar a causa raiz exata; 3/1800 na amostra): `unexpected token ";"` numa família de fontes de um único fornecedor (comércio exterior).
- Fixture de regressão: `tests/parser_gaps3_test.prw` (`cmd/advplc/parser_gaps3_test.go`, `TestParserGaps3Fixture`) — separa construções "parseadas e descartadas" (só precisam compilar) das que têm semântica real e precisam rodar certo.

Gaps encontrados numa varredura fresca e não-viesada de 1800 arquivos aleatórios dos dois corpora reais (811R4 + 12.1.2510), fora da amostra já 100%-curada.

Isolados com repro mínimo antes de cada correção — ver `tests/` para os casos.

- `BEGIN REPORT QUERY <alias> ... END REPORT QUERY <alias> [PARAM ...]` (wrapper de TReport em torno de `BeginSql/EndSql`) reconhecido pelo preprocessador e removido (linha em branco, preserva numeração de diagnóstico) — antes, essas duas linhas sobravam como código bruto depois do `BeginSql/EndSql` interno virar AdvPL, e a vírgula de `PARAM a, b` quebrava o parser.
- `&expr` (macro substitution) agora executa de verdade em runtime — bug real descoberto no processo: o VM emitia `OP_MACRO` no compile mas não tinha handler pra esse opcode, então qualquer `&cVar` / `&(expr)` executado (não só compilado) derrubava o processo com `unknown opcode: UNKNOWN`. Corrigido com um motor de macro-eval real (`pkg/vm/macro.go`): lexa+parseia+compila a string em runtime como uma expressão isolada e executa contra o mesmo `dynEnv` (Private/Public) do VM corrente — Locals não são visíveis, mesma limitação do Clipper real (Locals não existem por nome em runtime).
- `ident¯o` colado (`K2&cSuf`) — composição clássica de nome de variável dinâmica do Clipper (concatena o identificador literal com o valor em runtime da macro pra formar OUTRO nome de variável) agora parseia e resolve via o mesmo motor de macro-eval, em vez de quebrar o parser com `expected ')', got '&'` dentro de sub-expressões parentizadas.

Servidor REST nativo (classe `WSRestServer`)

O AdvPP agora executa REST 2.0 de verdade para o estilo de anotações do TLPP — até aqui `@Get` / `@Post` / `@Put` / `@Patch` / `@Delete` eram só sintaxe reconhecida e descartada (sem servidor HTTP nem despacho real), mesmo gap documentado no `MCPServer` v1.5.0. Mesmo padrão arquitetural: um `net/http.Server` real, sem CGO, sem dependências externas.

- `pkg/rest`: servidor HTTP em Go puro — roteador nativo do `net/http` (Go 1.22+, patterns `"GET /clientes/{id}"`), path params via `r.PathValue`, JSON de request/response via `encoding/json`; endpoint `GET / routes` de introspecção.
- Classe `WSRestServer` (`pkg/vm/rest_native.go`): `New(cNome, cVersao)` varre o bytecode carregado e auto-registra como rota toda `User Function` anotada com `@Get("/path")` / `@Post(...)` / `@Put(...)` / `@Patch(...)` / `@Delete(...)` (a anotação já chegava intacta até `FunctionInfo.Annotations` desde o suporte a namespaces — só faltava alguém ler); `AddRoute(cVerbo, cPath, cNomeFuncao)` registra rota manualmente. Cada requisição roda a função numa VM isolada (mesmo mecanismo do `MCPServer` / `StartJob`) recebendo um objeto único (`oParam:CAMPO`) com path params + query string + corpo JSON mesclados (corpo vence path, path vence query). Retorno vira JSON 200;

erro vira 500; path desconhecido 404; verbo não registrado num path existente 405. `Serve(nPorta)` bloqueia servindo HTTP.

- **Limitação conhecida, documentada e não implementada:** o DSL

clássico `WSRESTFUL <nome> ... WSMETHOD <verbo> PATH "..." ... ENDWSRESTFUL` continua só parseado — `parseWSClient`

(`pkg/parser/parser.go`) descarta o verbo HTTP e a cláusula `PATH` ao montar o `ast.ClassDecl`, e a implementação do método é ligada a uma instância de classe (`WSMETHOD ... WSSERVICE <classe>`), enquanto `v.RunFunction` — usado pelo dispatch HTTP — só chama funções top-level. Cobrir esse estilo exigiria capturar verbo+path no parser (cirurgia isolada, viável) e um caminho de dispatch novo que instancie o serviço a cada requisição antes de chamar o método (mudança maior na VM); nenhum dos dois casos apareceu nos corpora reais validados, então ficou fora desta rodada — workaround: escrever o serviço no estilo anotações, ou registrar a rota manualmente com `AddRoute`.

- Fixture `tests/rest_server_test.prw` + teste de integração Go (`cmd/advplc/rest_integration_test.go`, `TestRestServerFixture`): builda o binário, sobe o servidor, faz requisições HTTP reais (GET, GET com path param, POST com corpo JSON, 404, 405) — mesmo padrão do `TestMCPServerFixture`.
- **Bug de implementação pego durante o TDD:** `rest.Server.Serve` chamava `buildMux()` (que precisa de `s.mu`) enquanto `s.mu` já estava travado para gravar `s.httpServer` — deadlock silencioso no primeiro request (o processo ficava vivo, bloqueado em `ListenAndServe`, mas nunca aceitava conexão). Corrigido montando o mux antes do lock.

Mais 7 gaps de parser/compilador (segunda varredura, 2000 fontes)

Continuação da varredura acima numa amostra maior (2000 fontes reais dos corpora 811R4 + Protheus 12.1.2510): 93,0% → 96,95% de aprovação em `advplc check`. Fixture de regressão: `tests/parser_gaps_v1201.prw` (`cmd/advplc/parser_gaps_test.go`).

- **NamedParam (ident := valor) fora de uma lista de argumentos de call quebrava o compilador** ("unsupported expression type: *ast.NamedParam") — o maior bucket da varredura, de longe. O parser (desde o v1.19) transforma `ident := expr` em qualquer posição de argumento de call num nó `NamedParam`, mas só `compileArgs` sabia compilá-lo; qualquer outro lugar que chamasse `compileExpr` diretamente sobre esse nó (ex.: o 2º/3º argumento de `IIF / IF`, que não passa por `compileArgs`) caía no caso `default` e crashava. O idioma real mais comum que dispara isso é usar atribuição como valor de um ramo de `IIF` (`IIF(cond, cVar := "A", cVar := "B")`), não parâmetro nomeado de verdade. Fix: `compileExpr` agora trata `*ast.NamedParam` como um `AssignExpr` comum (atribui e deixa uma cópia na pilha) quando encontrado fora de `compileArgs`, em vez de crashar.
- **ACTIVATE POPUP/WINDOW/... AT nRow, nCol não era reconhecido** — a cláusula de posição do `ACTIVATE` só cobria `ON INIT / VALID / CENTERED`; `AT` ficava sobrando como um statement solto inválido, quebrando o parse do resto do arquivo.
- **RELEASE OBJECTS a, b (plural) não era reconhecido** — só a forma singular `RELEASE OBJECT` estava na lista de dispatch, apesar de a lógica de parsing já suportar lista separada por vírgula.
- **@ ... LISTBOX ... FIELDS ALIAS cAlias ... quebrava** — a cláusula `FIELDS` tratava incondicionalmente o que vier depois como o início de uma lista de valores (`FIELDS "a", "b", "c"`), então `ALIAS` virava o primeiro item dessa lista em vez de introduzir sua própria sub-cláusula de expressão única, dessincronizando o resto do parse do comando `@`.

- `DEFINE SECTION ... TABLE "a", "b"` (singular) não era reconhecido — só `TABLES` (plural) estava na lista de cláusulas de `DEFINE`; a variante no singular é o mesmo idioma tolerado pelo Clipper real.
- `@ ... METER ... BARCOLOR c1, c2` quebrava o parse do comando `@` inteiro — qualquer cláusula não cadastrada em `isAtClauseWord` interrompe o loop de cláusulas do `@` (não só é ignorada — o comando inteiro termina ali), sobrando o resto como statement inválido. `BARCOLOR` (cor da barra do controle METER) não estava na lista.
- `Default alias->campo := valor` quebrava — `Default` só aceitava identificador simples, `::prop` ou `Self:prop` como alvo; a forma `HttpGet->Page` (idioma comum do DSL web do Protheus, tratando o dicionário de querystring/form como pseudo work-area) não tinha tratamento, nem como primeiro nome nem como item subsequente de uma lista `Private a:=1, M->campo:=2` separada por vírgula (só o primeiro nome da lista tinha o tratamento de redundância `M->campo`, faltava nos itens seguintes).

Investigado e NÃO é gap real (falso positivo de header/include ausente)

- Comando DSL de projeto (ex.: `QSPARENTFIELD ... INDEX ORDER n`) definido via `#xcommand` num header próprio (`.ch`) que não está presente no corpus baixado — sem ele, o comando nem existe para o compilador real, então não há sintaxe nativa a suportar.
- Um `#include` de header presente no corpus mas fora do diretório-padrão de busca (`advplc check` só adiciona o diretório do próprio arquivo, não subpastas irmãs tipo `Includes/` do projeto original) — resolvível com `-I`, não é gap de linguagem.
- Um caso de encadeamento `x += y += z` (dois `+=` seguidos na mesma expressão) — bug de digitação no fonte original, não uma forma válida de atribuição composta em Clipper/AdvPL.

Parser — nova rodada de gaps de linguagem contra corpus real

Continuação da caçada de gaps de linguagem (rodada anterior corrigiu BEGIN REPORT QUERY, macro-eval em runtime, NamedParam fallback, servidor REST via WSRestServer, e 7 fixes de parser). Amostra de 2500 arquivos: 92,1% → 97,9% de aprovação; amostra nova de 1500: 96,3%. Todos os fixes são causas raiz reais confirmadas com repro mínimo isolado, não sintaxe especulativa.

- `NamedParam ( ident := expr )` fora de `compileArgs` também no curto-circuito de `IF()` / `IIF()`: o ramo de 3 argumentos chama `compileExpr` direto nos ramos (pra não avaliar os dois), sem passar por `compileArgs` — outro caminho, além do já corrigido na rodada anterior, que caía no `default` de `compileExpr`. Um argumento como `IIf(lConfirma := Func(...), a, b)` (atribuição real usada como argumento, idioma comum em AdvPL) virava "unsupported expression type: *ast.NamedParam".
- Atribuição (`:=`) como condição em `ElseIf` / `Case` / coordenada do `@`: `If` / `While` / `Return` já suportavam `x := cond` inline; `ElseIf`, `Case` do `Do Case` e a 1ª/2ª coordenada do comando `@ x, y VERB` não. Unificado via `parseAssignRHS`.
- `@ident := expr` sem coordenadas: `@` antes de um alvo de atribuição comum (idioma real onde `@` é só um marcador tolerado pelo compilador Clipper/AdvPL) travava o parser esperando uma 2ª coordenada.
- Atribuição composta (`+=` / `-=` / `*=` / `/=`) como lado direito de outra atribuição: `x := y += z` (ex.: `nIniDim := aCol[i] += valor`) — `parseAssignRHS` só recursava em `:=`, não nos compostos.
- `ACTION` / cláusulas de `DEFINE` e `MENUITEM` com lista de expressões: `ACTION cVar := Func(...), Outro(...)` (sequência estilo corpo de codeblock, com atribuição) não era aceito — só uma expressão simples.
- Variável-alvo do `DEFINE` depois da 1ª cláusula: `DEFINE SBUTTON FROM x, y oButton TYPE n ACTION ...` (variável depois de `FROM`, não logo após o kind) terminava o `DEFINE` cedo e confundia o resto da linha com um statement novo.
- `SET FILTER TO` vazio antes de statement com continuação `;`: a heurística de

"tem valor?" do `SET ... TO` aceitava um identificador solto como valor mesmo em outra linha lógica — e `DEFINE` não está na tabela de palavras-reservadas deste lexer (é um IDENT comum), então `Set Filter To` \n `Define MSDIALOG ...` engolia a palavra `DEFINE` inteira como "valor" do SET, quebrando tudo depois. Agora exige mesma linha.

- **DEFINE SECTION / CELL** — cláusulas `TABLE`, `BREAK`, `VAR`, `ID`, `DIALOGS`, `FIELDS`, `SIZES`, `ITEMS`, `HEAD`: TReport/REDEFINE usam bem mais cláusulas do que as poucas já suportadas; `BREAK` em particular (`PAGE BREAK` / `CELL BREAK` / `LINE BREAK`) é sempre um flag, nunca uma cláusula com valor — tentar ler um valor pra ela engolia o statement seguinte inteiro.
- **REDEFINE** (sem suporte nenhum antes): mesma DSL de cláusulas do `DEFINE`, usada pra vincular um controle a um recurso de dialog já existente. Reaproveita o parser do `DEFINE`; ganhou também suporte a alvo indexado (`DEFINE/REDEFINE BUTTON aoBotton[nX] ...`).
- **RELEASE OBJECTS** (plural) além de `RELEASE OBJECT`.
- **ACTIVATE MENU / ACTIVATE POPUP**: cláusulas `AT linha,coluna` e `WINDOW <expr>`.
- **LOCATE REST FOR**: `REST` não era reconhecido como flag, então o `FOR` da cláusula caía no `parseFor()` de verdade (statement de loop), que espera `FOR var := ...` e quebrava em `FOR (cond)`.
- **CREATE SCOPE <var> FOR <cond> [WHILE <cond>]** (sem suporte nenhum antes) — mesma causa raiz do LOCATE REST FOR.
- **DEFAULT** com alvo `alias->campo` ou indexado (`arr[i]`): só aceitava identificador simples ou `::Prop` / `Self:Prop`.
- **String delimitada por colchetes ([texto]) span multi-linha via continuação ; :** o lookahead da string de colchete só olhava até o fim da linha FÍSICA; strings longas legíveis com `;` de continuação (uma por linha) quebravam. Agora cruza linhas continuadas, igual ao tokenizer de `;` já faz pra statements.
- **@ x,y TO ... no nível top-level confundido com anotação @Nome**: o dispatcher de anotações de topo (`@Get`, `@Post(...)`) não distinguia de um statement de desenho de tela legado (`@0,0 TO 380,520 DIALOG ...`, comum em fontes sem `Function` no topo).
- **WSMETHOD ... PRODUCES x RESPONSE y**: `RESPONSE` (objeto de retorno) não era consumido depois de `PRODUCES` / `CONSUMES`, quebrando o resto da declaração `WSRESTFUL`.
- **WSMETHOD ... WSRECEIVE nome1,nome2,...**: nomes de parâmetro de negócio que colidem com palavras-chave do DSL `@ / DEFINE` (ex. `Type`, `Size`) cortavam a lista cedo demais — o parser de lista genérica usado ali reaproveitava por engano a heurística de "onde uma lista de cláusulas do `@` termina".
- **:MethodName(args) sem receiver**: idioma antigo Clipper/Class(y) de chamar a versão da superclasse de um método (comum na 1ª linha de `METHOD New(...) CLASS X`). Sem dispatch de herança real, tratado como `::MethodName(args)`.
- **NO VSCROLL / NO HSCROLL** como flags de duas palavras no DSL `@` (já existiam `NO SCROLL` / `NO UNDERLINE` / `NO BORDER`).
- **Replace <alias>-><campo> With <expr>**: o comando `REPLACE ... WITH` só reconhecia campo como um único token; via alias (`SE1->E1_CAMPO`, 3 tokens) não batia o gate e o comando inteiro era perdido.
- Investigado, sem correção clara (baixo impacto — 2 ocorrências em ~4000 arquivos, sem repro isolável): `expected TO in For` num `For var := n To expr` sintaticamente padrão — o repro isolado do trecho reportado passa limpo, indicando um erro real mais cedo no mesmo arquivo mascarado pelo drift de posição do pré-processador.
- Confirmado FALSO POSITIVO (não é gap de linguagem): comandos de projeto `QSPARENTFIELD` / `QSFIELD` / `QSTABLE` (DSL de `#xcommand` definido num header `.ch` específico do projeto, não incluído no corpus baixado) e uma construção obscura `IIf(cond, varA, varB) := valor` (atribuição condicional de lvalue via `IIf`, uma única ocorrência real, duplicada em dois diretórios do mesmo corpus).
- Fixture de regressão: `tests/parser_gaps2_test.prw` + `cmd/advplc/parser_gaps2_test.go` (`TestParserGaps2Fixture`), um caso mínimo por padrão corrigido acima.

[1.20.0] — 2026-07-22

Completa a álgebra linear do kernel matemático: **SVD** e **autovalores de matriz não-simétrica** (com pares complexos), e corrige o parser para compilar fontes Protheus reais (`=` em argumento é comparação, não parâmetro nomeado).

Álgebra linear — SVD, autovalores não-simétricos + correção de parser

- **SVD()** (decomposição em valores singulares por Jacobi de um lado): `{U, S, V}` com `A ≈ U · diag(S) · VT`, `S` decrescente, suporta matrizes retangulares.
- **Eig()** — autovalores de matriz **não-simétrica** real (redução a Hessenberg + QR de duplo shift à la EISPACK/hqr), devolvendo `{reais, imag}` com **pares complexos conjugados** corretos. VM: `oA:SVD()` e `oA:Eig()`. Fecham os follow-ups do S6b.
- **Correção de parser:** `ident = expr` como argumento de chamada agora é **comparação** (semântica AdvPL do `=`), não parâmetro nomeado — só `ident := expr` é named param. Corrige a compilação de fontes Protheus reais (ex.: `If(cAlias1 = cAlias, .T., .F.)`); `tests/real_protheus_test.prw` passa a compilar. Mensagem de erro de codegen agora inclui a linha.

[1.19.0] — 2026-07-22

Kernel matemático completo em AdvPP: precisão dupla selecionável no Tensor (S6a), álgebra linear (S6b: LU/Det/Solve/Inv/QR/EigSym), geometria espacial (S6c) e aritmética/estatística (S6d). O ML segue rápido em float32; o float64 entra sob demanda para cálculo exato.

Kernel matemático — Geometria + Aritmética/Estatística (Sub-projetos 6c/6d)

- **Geometria espacial** (`pkg/vm/geometry_native.go`): vetores/pontos 2D/3D (arrays AdvPL, float64) — `VecDot`, `VecCross` (3D), `VecNorm`, `VecNormalize`, `VecDist`, `VecAngle`, `VecAdd` / `VecSub` / `VecScale`, `RotateVec2`, `RotateVec3` (eixo x/y/z).
- **Aritmética + estatística** (`pkg/vm/mathstat_native.go`): `Atan2`, `Log10`, `Pow`, `Ceil`, `Sign`, `Sinh` / `Cosh` / `Tanh`, `Gcd`, `Lcm`, `Fact`; `Mean`, `Variance`, `StdDev`, `Median`, `LinReg` (mínimos quadrados), `Interp` (linear). Fecham o kernel matemático (S6a f64 → S6b álgebra → S6c geometria → S6d aritmética/estatística).

Kernel matemático — Álgebra linear (Sub-projeto 6b)

- **Álgebra linear float64** (`pkg/tensor/linalg.go`, não-diferenciável): decomposição **LU** (pivô parcial) e, sobre ela, **Det** (determinante), **Solve** (`A · x = b`, `b` vetor ou matriz) e **Inv** (inversa). **QR** por refletores de Householder (`{Q,R}`), e **EigSym** (autovalores/autovetores de matriz simétrica por rotações de Jacobi, `{valores, vetores}`). VM: `oA:Det()/Solve(oB)/Inv()/QR()/EigSym()`. Erros (não-quadrada, singular, não-simétrica) são `ErrorValue`. SVD e autovalores não-simétricos ficam para um ciclo posterior.

Kernel matemático — Tensor float64 (Sub-projeto 6a)

- **Precisão dupla selecionável por tensor** no kernel de Tensor: `DType` (`float32` default / `float64`) com armazenamento `Data64 []float64`; acessores neutros `Get` / `Set`, construtores `NewDType` / `FromData64` / `RandDType`, `AsDType` / `SameDType`. Ops `Add` / `Sub` / `Mul` / `Div` / `MatMul` / `Transpose` / `Reshape` ganham caminho float64 por **dispatch no topo** (o caminho float32 fica byte-idêntico — o ML não é afetado) e **promovem a float64** se qualquer operando for f64. Novos `Dot` (produto interno) e `Norm` (L2). VM: `Tensor():New(aForma [, cDType])`, `FromArray(...[, cDType])`, `DType()`, `ToFloat32()` / `ToFloat64()`, `Dot` / `Norm`. Base do kernel de álgebra linear (S6b) e geometria (S6c). Propagação de f64 pelo autodiff adiada (álgebra/geometria são não-diferenciáveis).

[1.18.0] — 2026-07-22

Modelo de código AdvPL orientado a desenvolvimento (`dev_nn` , token-level) + biblioteca de 25 algoritmos, e três correções na VM/compilador descobertas ao construí-lo (relacional de string, `HasProperty` , `NamedParam`).

Correções na VM / compilador

- **Operadores relacionais em strings agora são lexicográficos.** `<` , `>` , `<=` , `>=` comparam strings por byte (via `strings.Compare`) quando ambos os operandos são strings; antes jogavam tudo em `ToFloat` ($\rightarrow 0$), fazendo `" " >= "A"` retornar `.T.` . Comparação numérica inalterada. (`pkg/vm/relcmp_test.go` .)
- **`JsonObject.HasProperty()` agora é case-sensitive**, consistente com o bracket (`oJ[k]`), `SetProp` , `DelName` e `GetNames` — todo o namespace de chaves JSON. Antes fazia `ToUpper` na chave, falhando para qualquer chave com letras (ex.: contar/deduplicar tokens com chave em variável).
- **`NamedParam ( ident := valor )` tratado em chamadas de self-método (`::m(...)`) e `New(...)` ,** roteando esses args por `compileArgs` como os demais calls.

Modelo de código AdvPL orientado a desenvolvimento (Sub-projeto 5)

- **`tests/llm/dev_nn.prw` —** LM neural de **código AdvPL, token-level**, treinado 100% em AdvPP sobre o stack de ML (nenhuma op de motor nova). Um **lexer AdvPL escrito em AdvPL** quebra a fonte em tokens (keywords, identificadores, números, strings, operadores multi/mono-char, `<nl>`); vocabulário top-N por frequência + `<unk>` (hash via `JsonObject`); mesmo NPLM do `pt_neural` (`Embedding` $\rightarrow$ `Reshape` $\rightarrow$ `Linear` $\rightarrow$ `Tanh` $\rightarrow$ `Linear` $\rightarrow$ `SoftmaxCE` , Adam via `Fit`) sobre tokens. Geração com des-tokenização + **REPL de autocomplete** (`ConIn`). No corpus real (46k tokens, vocab 301) a loss cai $\sim 5.70 \rightarrow \sim 0.31$, gerando idiomas AdvPL.
- **`tests/llm/algos_advpl.prw` —** biblioteca de **25 algoritmos** de lógica/leetcode/script em AdvPL puro (ordenação, busca, recursão, strings, DP: troca de moedas e LCS, Kadane, two-sum, FizzBuzz, parênteses balanceados...), cada um testável, com auto-teste. Dá o viés algorítmico ao corpus do `dev_nn` .
- **`tests/llm/build_corpus.sh` —** monta `code_corpus.txt` concatenando os fontes AdvPL/TLPP do repo (derivado; fora do git).
- Teto honesto documentado: modelo pequeno em VM interpretado — gera código plausível enviesado a lógica, não resolve problemas novos de leetcode.

[1.17.0] — 2026-07-22

Capstone da missão original: um **LM neural treinado por gradiente, 100% em AdvPP**, sobre o stack de ML lançado na v1.16.0. Adiciona a op `Reshape` diferenciável e o modelo `pt_neural.prw` (tokenizar $\rightarrow$ treinar $\rightarrow$ gerar), e reúne os modelos de LM em AdvPL puro em `tests/llm/` .

LM neural char-level treinado por gradiente (Sub-projeto 4)

- Op **`Reshape` diferenciável** no autograd (`Variable:Reshape(aShape)`): forward via `tensor.Reshape` , backward reshapa o gradiente de volta à forma original. Método na VM + grad-check por diferenças finitas (`go test`) + fixture `tests/reshape_test.prw` . É a peça que faltava para concatenar os embeddings de contexto.
- **`tests/llm/pt_neural.prw` —** o **capstone**: LM neural char-level (byte-level, seguro para UTF-8) treinado 100% em AdvPP sobre o stack S2+S3. NPLM estilo Bengio: `Embedding` $\rightarrow$ `Reshape` $\rightarrow$ `Linear` $\rightarrow$ `Tanh` $\rightarrow$ `Linear` $\rightarrow$ `SoftmaxCE` , treinado com **Adam via `Fit`** ; geração char-a-char por amostragem com temperatura. Tokenizador por código de byte (roundtrip exato de acentos), montador de exemplos por janela deslizante com amostra por stride. Auto-teste determinístico (mini-corpus): loss $\sim 2.77 \rightarrow \sim 0.04$. No `corpus.txt` real (*Dom Casmurro*, vocab 97): loss $\sim 4.58 \rightarrow \sim 0.06$, gerando morfologia

PT-BR. É o primeiro LM do projeto que **aprende os pesos por backprop**.

- **Organização:** os modelos de LM em AdvPL puro (`pt_llm` , `pt_chat` , `pt_nn` , `pt_neural`) e o `corpus.txt` foram reunidos em `tests/llm/`.

[1.16.0] — 2026-07-22

Ciclo de ML: núcleo de Tensor float32 + motor de autodiff completo + treino. Fecha os Sub-projetos 2 (Tensor forward) e 3 (autodiff/treino: S3a motor+SGD → S3b softmax-CE+Adam+embedding → S3c módulos+trainer), permitindo definir e treinar um modelo float de verdade em poucas linhas de AdvPL, com o Go fazendo a conta.

Núcleo de Tensor (Sub-projeto 2, forward)

- Classe `Tensor` (float32) acelerada em Go (`pkg/tensor`): dados `[]float32` planos fora da representação *boxed* de `Value` , com kernels de forward em Go puro — `MatMul` /matvec, elementwise com broadcast limitado (escalar, linha, coluna), `Transpose` , `Reshape` , reduções (`Sum` / `Mean` / `Max` / `Argmax` , com e sem eixo), ativações (`Exp` / `Log` / `Sqrt` / `Relu` / `Tanh` / `Sigmoid` / `Gelu`), `Softmax` estável e `IndexRows` (embedding). O AdvPL orquestra; o Go faz a conta. Ligada à VM no padrão de classe nativa (`ObjectValue.Native`); erros de forma são `ErrorValue` capturáveis por `Try/Catch` . Aceite: `tests/mlp_demo.prw` roda o forward de um MLP float. Autodiff/treino fica para um ciclo futuro.

Autodiff + treino (Sub-projeto 3a)

- Motor de **diferenciação reversa** (`pkg/autograd`): classe `Variable` (valor + grad + tape de ops) e `Backward()` em ordem topológica reversa; ops diferenciáveis `MatMul`, `Add` (broadcast), `Mul`, `Relu`, `Sum`, `Mean` e a loss `MSE`, todas reusando os kernels do `pkg/tensor` (S2, intocado). Otimizador **SGD** (`Step` / `ZeroGrad`). Classes AdvPL `Variable` e `SGD` . Corretude por verificação numérica de gradiente (diferenças finitas) no `go test` ; aceite `tests/train_demo.prw` treina um MLP e a loss cai bem abaixo da inicial. `Softmax-CE`, `Adam`, `embedding` e módulos ficam para os próximos ciclos.

Loss de LM + Adam (Sub-projeto 3b)

- `Autograd` ganhou a loss de classificação `SoftmaxCE` (softmax + cross-entropy fundida, estável; gradiente `softmax - onehot`), o otimizador **Adam** (com correção de viés), o backward de **embedding** (`IndexRows` diferenciável, scatter-add) e das ativações **Tanh/Sigmoid/Gelu**. Classes AdvPL: métodos novos em `Variable` + classe `Adam` . Aceite: `tests/classifier_demo.prw` treina um classificador (loss cai, acurácia 100%). Módulos e trainer ficam para o S3c.

Módulos e trainer (Sub-projeto 3c)

- Módulos de rede neural (`pkg/nn`): `Linear` (W,b) e `Embedding` (tabela), cada um encapsulando seus parâmetros + `Forward` e expondo `Params()` para o otimizador. Native `Fit(bPasso, nEpocas)` roda o laço de treino avaliando um codeblock por época (usando closures). Classes AdvPL `Linear` / `Embedding` . Aceite: `tests/nn_demo.prw` define e treina um MLP com módulos + Adam em poucas linhas (acurácia 100%). Fecha o "Full autodiff/treino" (S3a motor → S3b LM loss/Adam → S3c módulos).

[1.15.0] — 2026-07-22

Robustez da linguagem (Sub-projeto 1)

- `If()` / `IIF()` **curto-circuito** — com 3 argumentos, avaliam só o ramo escolhido (forma especial no compilador); antes o native avaliava os dois ramos.

- **GetNames(oJson)** — itera as chaves de um `JsonObject` na ordem de inserção (native e método OO), com `ObjectValue` mantendo a ordem via `SetProp / DelProp`.
- **Closures aninhadas** — upvalues tipados (LOCAL/UPVAL) com resolução recursiva; um codeblock dentro de codeblock captura Locais N níveis acima por referência.
- **Private / Public com escopo dinâmico** — variáveis dinâmicas visíveis às funções chamadas até o declarante retornar (ambiente dinâmico na VM + `OP_LOAD_DYN / OP_STORE_DYN / OP_DECL_DYN`, restauração em todos os caminhos de unwind). Nome não declarado dentro de função passa a resolver dinamicamente (semântica AdvPL correta).
- **Try/Catch cross-frame** — uma exceção lançada numa função mais funda que o `Try/Catch` agora é capturada no nível certo (antes abortava o programa); o desenrolamento de pilha restaura os bindings dinâmicos corretamente.

Desenvolvido via pipeline spec → plano → execução por subagentes com revisão em dois estágios por task e revisão final de branch inteira (ver [docs/superpowers/](#)).

Known-issues (não bloqueiam; documentados): a forma especial `If / IIF` sombreia uma `User Function If() / IIF()` homônima de 3 argumentos (AdvPL reserva esses nomes); `Try/Catch` **aninhado na mesma função** captura no handler errado e `Finally` não roda no caminho de sucesso — ambos **pré-existent**s, não introduzidos neste ciclo.

[1.14.1] — 2026-07-22

- **Corpus externo grande de verdade:** `corpus.txt` agora é *Dom Casmurro* de Machado de Assis (domínio público, via Project Gutenberg) — ~72 mil tokens, patamar expressivo. O pipeline treina sobre ele em ~30s (vocabulário capado em top-N + `<unk>`, treino por stride, bag long-context incremental). A saída fica temática/machadiana (Capitu, Escobar, Bentinho, "olhos de ressaca"); nota honesta: prosa literária complexa excede a capacidade de um modelo n-grama+ELM, então o texto é temático mas não totalmente coerente — o corpus factual curado (embutido em `Corpus()`, usado como fallback) gera prosa mais limpa.
- **Penalidade de repetição em janela** (`pt_nn.prw`): pune palavras repetidas nas últimas REPWIN posições, eliminando loops de palavras funcionais ("se se se"). Melhora qualquer corpus.

[1.14.0] — 2026-07-22

Evolução do LLM híbrido para **janela longa (entrada e saída até 4096 tokens cada)** e funções nativas que faltavam, implementadas no compilador conforme necessário.

Novas funções nativas / correções na VM

- **Funções de array de ordem superior com bloco de código** — `ASort` (era no-op quanto ao comparador), `AEval` (era no-op) e `AScan` (era só por valor) agora avaliam de verdade um `{ [...] ... }`. Implementadas com um novo helper `callBlockSync` na VM, que roda um bloco de forma síncrona no VM atual (loop aninhado até o frame do bloco retornar). Limitação conhecida: blocos não capturam Locais externos (closures reais) — usar só os parâmetros do bloco.
- **File(cArq)** — checagem real de existência via `os.Stat` (era um stub que retornava sempre `.F.`).

LLM híbrido — janela longa

- `pt_nn.prw` evoluído para **janela longa**: contexto local posicional + **bag long-context** sobre uma janela de até 4096 tokens (mantido incrementalmente, O(1) amortizado por token); aceita seed de até 4096 tokens e

gera **documento multi-frase** de até 4096 tokens. Algoritmos modernos: perceptron médio (Collins 2002), suavização interpolada (Jelinek-Mercer), amostragem nucleus (top-p), vocabulário limitado por frequência (top-N + `<unk>`) e amostra de treino por stride (custo do treino independente do tamanho do corpus). Carrega `corpus.txt` grande via `MemoRead` se presente.

[1.13.0] — 2026-07-21

BLAS ternária, console interativo e três modelos de IA escritos inteiramente em AdvPL — culminando num LLM híbrido Markov + rede neural ternária que treina e infere sem multiplicação, sem sair da linguagem.

Novas funções nativas

- `MatVecTern(aMat, aVecTern)` (`pkg/vm/natives.go`) — BLAS ternária: produto matriz-vetor *multiply-free* onde o vetor é ternário (`-1 / 0 / +1`), `result[i] = Σ_j sign(vec[j])·mat[i][j]` (só soma/subtração, o kernel do BitNet exposto ao AdvPL). Base para redes neurais ternárias sem BLAS de ponto flutuante nem GPU.
- `ConIn([cPrompt])` (`pkg/vm/vm.go`, `pkg/vm/natives.go`) — lê uma linha do stdin (contraparte de `ConOut`), viabilizando programas de console interativos (REPL, chat). Leitor bufferizado por VM, inicializado sob demanda.

Modelos de IA em AdvPL puro (exemplos)

- `pt_chat.prw` — respondedor por recuperação (retrieval): normaliza a pergunta (minúsculas + remoção de acentos), tokeniza, descarta stopwords e pontua uma base de conhecimento por sobreposição de palavras-conteúdo. Lê e **responde** em PT-BR, num REPL via `ConIn`, com fallback educado.
- `pt_nn.prw` — LLM híbrido **Markov + rede neural ternária** (Extreme Learning Machine): camada escondida = projeção aleatória ternária fixa com contexto posicional (via `MatVecTern`), ativação ternária `sign(...)`, e camada de saída treinada por **perceptron multiclasse** (puro add/sub, sem gradiente/float). A geração mistura Markov (prior local, por frequência) com o rerank aprendido da rede, com anti-repetição e amostragem com sharpening. Treina e infere 100% em AdvPL; o aprendizado é medível (erros do perceptron caem ~85% ao longo das passadas). Corpus embutido, escalável via `MemoRead`.

Testes

- Fixture `tests/ternary_ai_test.prw` exercitando `MatVecTern` e `ConIn`.
- Suíte Go verde; `make test` com os fixtures novos.

[1.12.0] — 2026-07-21

Fechamento de lacunas da VM/compilador descobertas ao escrever um LLM de Markov em AdvPL puro (`pt_llm.prw`): três bugs de controle de fluxo e a ausência de I/O de disco e chamada de sistema.

Correções de controle de fluxo

- `Loop` (continue) e `Exit` (break) agora funcionam — `pkg/compiler/codegen.go`. Antes, ambos emitiam um `OP_JUMP` para sentinelas `-998 / -999` que **nunca eram corrigidas**, então `Loop` estourava a VM com índice de instrução negativo e `Exit` saltava para lixo. Adicionada uma pilha de `loopContext` que acumula os jumps de break/continue de cada loop e os corrige no fim de `compileFor / compileWhile` (continue → incremento/condição, break → fim do loop). Funciona com loops aninhados — `Exit` quebra apenas o loop interno.
- `For ... Step` negativo (descendente) agora itera — novo opcode

`OP_FORLOOP_CMP`. A condição de continuação era fixa em `var <= end` (`OP_LTE`), então qualquer `For i := N To 1 Step -1` executava zero vezes. O novo opcode recebe `[var, end, step]` e escolhe `<=` ou `>=` pelo sinal do **step em tempo de execução** — cobre `Step -1`, `Step -2` e até step com valor variável. Como efeito colateral, o step passou a ser avaliado uma única vez (numa local escondida), em vez de reavaliado a cada iteração.

Correção de semântica de hash

- **Chaves de `JsonObject` por bracket agora são case-sensitive** — `pkg/vm/vm.go`. `OP_ARRAY_GET` / `OP_ARRAY_SET` aplicavam `strings.ToUpper` na chave, colapsando `h["Brasil"]` e `h["brasil"]` na mesma entrada e inutilizando o objeto como dicionário. Removido o upper apenas do acesso por bracket; o acesso por ponto (`obj.Prop`) continua case-insensitive (semântica correta de objeto AdvPL). Efeito colateral positivo: `toJson()` agora preserva o case original das chaves.

Novas funções nativas — I/O de disco, arquivo e sistema

- **I/O de disco:** `MemoRead(cArq)`, `MemoWrite(cArq, cTexto)` (+ alias `MemoWrit`), `Ferase(cArq)`.
- **API de handle de arquivo (streaming):** `FCreate`, `FOpen`, `FReadStr`, `FWrite`, `FSeek`, `FClose`, `FError` — com tabela de handles na VM (`fileHandles map[int]*os.File`). Permite ler/gravar arquivos grandes em blocos sem carregar tudo na memória. A leitura usa `FReadStr` (retorna a string) em vez de `FRead` com buffer por referência, porque os natives da VM recebem valores, não lvalues.
- **Chamada de sistema:** `WaitRun(cCmd)` executa no shell do SO (cross-platform `sh -c` / `cmd /c`), herda stdio, espera e retorna o exit code. A captura de stdout é feita pelo padrão AdvPL de redirecionar para arquivo e ler com a API de handle (funciona em streaming, para saída grande).

Exemplo novo

- `pt_llm.prw` — modelo de linguagem de Markov de ordem variável (nível de byte, ordens 1–6 com backoff) escrito inteiramente em AdvPL, que lê e gera português do Brasil. Roda com `advplc run pt_llm.prw`, com auto-teste incluído.

Testes

- Suíte Go completa verde (`go test ./...`): `pkg/compiler`, `pkg/vm` (via `cmd/advplc`), `pkg/runtime` e todos os demais pacotes. Fixtures `.prw` novos cobrindo os três bugs de loop, o case-sensitivity, e as APIs de I/O/handle/sistema.

[1.11.0] — 2026-07-11

Debugger real via DAP (breakpoints/step/variáveis) — launch e attach

Pedido do usuário: testes de compilação/debug completos antes de estender o mesmo suporte ao plugin de VS Code na frente web. Resultado: `advplc` ganha um adaptador completo de Debug Adapter Protocol, cobrindo os dois modos de execução que o compilador já suportava.

- `advplc debug` (modo *launch*, sobre stdio): compila e executa um único fonte com breakpoints, step over/in/out, call stack e inspeção de variáveis locais. Implementado como hook por instrução na `runLoop` da VM (`pkg/vm/debug.go`) — usa a linha-fonte que cada instrução de bytecode já carregava e o call-frame stack já existente, custo zero quando nenhum debugger está anexado (`if v.debugger != nil`). Servidor DAP em

`pkg/dap/`, stdlib puro, mesmo padrão de framing do LSP

(Content-Length). `os.Stdout` é redirecionado para `/dev/null` dentro do adaptador — `ConOut` / `ConOutW` sempre espelham no stdout real, que aqui é o próprio transporte DAP; sem isso o primeiro `ConOut()` do programa depurado corrompia o stream JSON no meio da sessão.

- `advplc serve --debug-port N` (modo *attach*): `advplc serve` é um processo de vida longa com uma VM por sessão de browser — arquitetura diferente de `debug`, que expõe um listener TCP DAP separado da porta HTTP. Cada nova sessão de browser oferece sua VM ao servidor DAP anexado (`dap.Server.OfferSession`), que só assume a VM depois que o cliente concluiu `attach` + `configurationDone` — deliberadamente **uma sessão depurada por vez** (sem multiplexação de threads DAP; uma segunda aba conectando durante uma depuração roda normal, sem debugger). Saída do console espelha simultaneamente no browser e no Debug Console do editor.
- Testado ponta a ponta duas vezes: cliente DAP bruto em Python (breakpoint bate na linha certa, variáveis corretas ANTES da instrução executar, call stack aninhado correto) e ao vivo pela GUI real do NeuralInverse/VS Code (toolbar mostra ícone de "disconnect" pra attach vs "stop" pra launch — o editor distingue os dois modos automaticamente).

Extensão VS Code (`advpl-tlpp`) publicada no Marketplace

`tools/vscode-advpl/` — antes só sideload via `.vsix`, agora publicada em marketplace.visualstudio.com/items?itemName=PederMunksgaard.advpl-tlpp.

Cobre: syntax highlighting completo, keybindings reais (`Ctrl+F9` build, `F9` run, `Ctrl+Shift+F9` compile, `Ctrl+Alt+F9` serve), e o debugger DAP (launch zero-config via `F5`, attach via comando dedicado). Motivo deste release: a extensão publicada já chamava `advplc debug / --debug-port`, que não existiam no compilador oficial (v1.10.3) — esse release fecha essa lacuna entre o que estava publicado no Marketplace e o que o compilador publicado sabia fazer.

Verificação antes do release

Sweep completo contra os 2 corpora reais (811R4 + Protheus 12.1.2510, 500 arquivos amostrados): 96,2% de aprovação, **as mesmas 19 falhas** que já existiam no v1.10.3 publicado — zero regressão introduzida. Cross-compile verificado para os 4 alvos oficiais (linux/amd64, linux/arm64, windows/amd64, darwin/arm64). Diff revisado linha a linha contra v1.10.3: mudanças estritamente aditivas, nenhum caminho existente (`run` / `compile` / `build` / `check` / `ast` / `bytecode`) alterado.

[1.10.3] — 2026-07-11

3 bugs reais do executável standalone no Windows, encontrados testando de verdade

Pedido do usuário depois da v1.10.2: "testar o binário standalone gerado no Windows também". Os binários das ferramentas (`advplc.exe`/`adveditor.exe`/`advpp-ide.exe`) já eram compilados nativamente no Windows pelo workflow de release, mas o RECURSO de gerar um executável standalone a partir de um programa AdvPL nunca tinha sido exercitado de verdade nesse SO — só localmente em Linux. Adicionado um step de CI dedicado (Windows apenas, `.github/workflows/test.yml`) que builda E RODA um executável standalone gerado a partir de um fixture simples (`tests/standalone_console_test.prw`) num runner `windows-latest` de verdade — não bastava só compilar, o teste efetivamente executa o binário e verifica a saída. Isso encontrou 3 bugs reais, um atrás do outro:

1. Mover o executável entre drives diferentes falhava: `os.Rename`

dá erro `"cannot move the file to a different disk drive"` no Windows quando origem e destino estão em volumes diferentes — exatamente o layout dos runners `windows-latest` do GitHub Actions (temp em `C:`, checkout em `D:`), plausível também em máquinas reais. Corrigido com fallback copy+remove (`moveFile`) quando o rename falha — cobre também o `EXDEV` equivalente no Unix para mounts diferentes.

2. **O fallback de cópia então falhava ao remover a origem:** o handle do arquivo de origem ainda estava aberto (só fechado via `defer`, que roda DEPOIS do `return`) quando `os.Remove` tentava apagá-lo — o Windows recusa apagar um arquivo com handle aberto (ao contrário do POSIX, que permite `unlink` em arquivo aberto). Corrigido fechando o handle explicitamente antes do `os.Remove`, não via `defer`.
3. **O executável gerado rodava corretamente (MSDIALOG funcionava, saída aparecia) mas nunca se fechava sozinho ao terminar** — ficava pendurado precisando ser morto à força. Rastreado com um trace opt-in (`ADVPP_STUB_TRACE=1`) instrumentando cada etapa do stub: `a.Quit()` (chamado pela goroutine de fundo da VM ao terminar) retornava normalmente, mas o event loop do Fyne/GLFW dentro de `w.ShowAndRun()` nunca notava o canal de encerramento fechado e nunca retornava — especificamente neste ambiente. Uma primeira tentativa (remover uma chamada duplicada a `Show()` antes de `ShowAndRun()`, que também já chama `Show()` internamente) não resolveu. Corrigido de forma mais robusta: como o executável standalone é um script de vida curta, não um app interativo de longa duração, não há necessidade de um handshake gracioso de fechamento de janela via Fyne — a goroutine agora chama `os.Exit(0)` diretamente assim que `v.Run()` termina, encerrando o processo incondicionalmente independente do que `w.ShowAndRun()` estiver fazendo. Caminho de erro inalterado: a janela continua aberta para o usuário ver o erro, fechando manualmente.

Cada um dos 3 bugs foi confirmado corrigido rodando de verdade no runner Windows do CI (não just localmente) antes de seguir para o próximo — o teste de CI também foi corrigido no meio do caminho para falhar corretamente quando o processo precisa ser morto à força (a versão anterior só conferia o conteúdo do log, deixando passar silenciosamente um hang que precisou de kill manual).

Sem regressão: `go build ./...`, `go vet ./...`, `go test ./...`, `make test` (31 fixtures, incluindo o novo fixture de smoke test), sweep completo do corpus de 500 arquivos (500/500) a cada commit.

[1.10.2] — 2026-07-11

MSDIALOG renderiza de verdade no advpp-ide (não só no modo web)

Pedido do usuário: testou o FrameworkClassesTest e o botão Compilar no advpp-ide e achou dois problemas reais. Investigação mostrou que o segundo era bem maior do que parecia: `DEFINE MSDIALOG / ACTIVATE MSDIALOG` (o DSL clássico `@ x,y SAY/GET/BUTTON`) e `FWMBrowse` dependem de interfaces opcionais (`DialogUI / BrowseUI`, `pkg/vm/dialog.go / pkg/vm/browse.go`) que só o renderer web (`advplc serve`) implementava — `pkg/ui.FyneUIProvider` (usado por advpp-ide) só tinha os 4 primitivos de mensagem (`MsgInfo / MsgStop / MsgAlert / MsgYesNo`). Qualquer programa com MSDIALOG literalmente **falhava** ("MSDIALOG: requer o modo web") ao rodar pelo IDE desktop.

- **FyneUIProvider.Dialog** (`pkg/ui/msdialog.go`, novo): implementa `vm.DialogUI` nativamente em Fyne — renderiza o spec (linhas SAY/GET

agrupadas pela heurística de grade y/x já existente no VM, botões no rodapé) como um `dialog.CustomDialog` de verdade, com um `chan dialogAction` bloqueando a goroutine chamadora até o clique de um botão (ou Escape/fechamento), devolvendo os valores digitados para writeback nas variáveis AdvPL — mesmo protocolo JSON que o renderer web já usa, então o VM não precisou mudar nada.

- **`v.Run()` precisou sair da goroutine principal do Fyne**
(`cmd/advpp-ide`, `run()`): como `Dialog()` bloqueia quem chamou até o clique do botão, e esse clique só é processado pelo event loop principal do Fyne, rodar `v.Run()` direto no handler do menu "Run" causava deadlock garantido assim que um programa abrisse um MSDIALOG. Corrigido rodando a VM em `go func() { v.Run() ... }()`.
- **Bug real encontrado de bônus, mesma causa raiz: `MsgYesNo` sempre devolvia `false` independente da escolha do usuário** —
`dialog.ShowConfirm` é assíncrono (o callback só roda depois), mas o código antigo lia a variável de resultado ANTES do callback rodar. Corrigido com o mesmo padrão de canal bloqueante — seguro agora que `v.Run()` não roda mais na goroutine principal.
- **`ConOut` /console não aparecia em lugar nenhum visível no IDE** — a VM escrevia direto em `os.Stdout` (sem terminal visível num app GUI empacotado) porque `run()` nunca chamava `v.SetOutputWriter`. Corrigido roteando para o próprio painel de saída do IDE.
- **Botão "Compile" agora gera um arquivo de verdade:** antes só reportava contagem de funções/classes e descartava o bytecode compilado. Agora salva `<arquivo>.bytecode` (mesmo formato que `advplc compile` já gerava), carregável depois via `advplc run` ou o próprio botão Run do IDE sem recompilar.
- **Novo item de menu "Build standalone executable...":** mesmo mecanismo de `advplc build`, extraído para `compiler.BuildStandalone` (`pkg/compiler/standalone.go`) e compartilhado entre CLI e IDE. Só funciona rodando de dentro de (ou apontando via `ADVPP_SRC` para) um checkout completo do código-fonte do AdvPP com o toolchain Go instalado — o stub gerado importa `pkg/compiler / pkg/vm` deste módulo, que não está publicado em lugar nenhum que `go build` consiga buscar sozinho; não é algo que funcione a partir de um pacote de release baixado isoladamente. Bug real corrigido de quebra: a detecção antiga da raiz do projeto (`filepath.Dir(filepath.Dir(caminhoDoFonte))`) só funcionava por coincidência para fontes dentro de `tests/` deste repo — quebrava para qualquer arquivo real de usuário. Substituída por busca robusta (sobe a árvore de diretórios a partir do cwd e do executável em execução, procurando o `go.mod` do módulo certo).
Sem regressão: `go build ./...`, `go vet ./...`, `go test ./...` (testes novos em `pkg/ui` para o contrato JSON do diálogo e em `pkg/compiler` para a busca de módulo), `make test` (30 fixtures), sweep completo do corpus de 500 arquivos (500/500). Verificado visualmente via Xvfb: MSDIALOG abre, campos são editáveis, clique em botão fecha o diálogo e grava os valores de volta corretamente nas variáveis AdvPL.

Executáveis standalone ganham UI Fyne e banco de dados completos

Pedido do usuário: quem compila um binário standalone via `advplc build` (ou o novo botão do advpp-ide) deveria conseguir usar TODAS as características da linguagem, não só a parte headless. O stub gerado (`pkg/compiler/stub_template.go`) rodava com `uiEnabled=false` e sem nenhum provider de UI nem banco de dados anexado — um programa usando

`MsgInfo`, `MSDIALOG`, `FWMBrowse` ou acesso a tabela simplesmente perdia essa funcionalidade quando virava um `.exe` /binário standalone, sobrando só a saída de console.

- O stub agora abre uma janela Fyne pequena que funciona ao mesmo tempo como console de saída (`ui.OutputConsole` , via `v.SetOutputWriter`) e como parent dos diálogos (`ui.FyneUIProvider`) — a MESMA dupla função que corrige de brinde um problema real no Windows: um binário Fyne gerado como GUI-subsystem não tem terminal anexado, então `ConOut` simplesmente desaparecia sem essa janela.
- Banco de dados conectado com o mesmo `SetDBFactory` / `ResolveDatabasePath` local (`./advpp.db`) usado por `advplc/adveditor/advpp-ide`.
- `pkg/ui.ConsoleWriter` extraído (antes privado em `cmd/advpp-ide`) para ser reaproveitado pelo stub também.
- Verificado visualmente via Xvfb: binário standalone gerado a partir de `tests/msdialog_test.prw` abre a janela, renderiza o MSDIALOG, grava os valores digitados e encerra sozinho ao fim da execução (`a.Quit()`), sem deixar processo pendurado; um programa 100% console (só `ConOut`) também abre a janela (double função de terminal), mostra a saída e encerra sozinho sem exigir interação do usuário.

FWMBrowse renderiza de verdade no advpp-ide + bug real de recno corrigido

Continuação do item anterior: `FWMBrowse` também dependia da mesma `BrowseUI` só implementada pelo renderer web.

- `FyneUIProvider.Browse` (`pkg/ui/browse.go` , novo): grid real (`widget.Table` , linha 0 = cabeçalho) com botões Novo/Editar/Excluir/Fechar, mesmo protocolo JSON (`browseSpec` / `browseAction`) que o renderer web já usa. Editar/Novo abrem `dialog.ShowForm` pré-preenchido; Excluir pede confirmação antes.
- **Bug real encontrado e corrigido em `pkg/vm/browse.go`** (compartilhado pelos dois renderers, não é algo introduzido agora): `browseItems` selecionava o pseudo-campo `rowid` sem apelido e procurava o resultado pela chave `"ROWID"` — mas o SQLite devolve o NOME da coluna de resultado usando o nome da própria coluna `INTEGER PRIMARY KEY` da tabela quando ela tem uma (`R_E_C_N_O_` , em toda tabela gerenciada pelo AdvPP desde a convenção de exclusão lógica da v1.10.1) — não literalmente `"rowid"`. Na prática, `recno` sempre voltava como 0, e todo "Editar" virava um INSERT duplicado em vez de um UPDATE. Corrigido apelidando a coluna explicitamente (`AS browse_recno`), o que torna o nome do resultado independente do schema da tabela.
- Verificado visualmente via Xvfb, ciclo completo: abrir browse com dados reais (SX3 + SA1), selecionar+editar um registro (grava no lugar certo, sem duplicar), Novo (insere), Excluir (soft-delete via `D_E_L_E_T_`), Fechar (retorna o controle ao programa AdvPL).

8 classes complexas do framework: adiadas por decisão explícita do usuário

Pedido original também citava `FWWizardControl`, `FWDynDialog`, `FWPanel`, `FWGroupBox`, `FWTabs`, `FWSplitter`, `FWTreeView`, `FWListView`. Antes de investir nelas, verificado o uso real nos dois corpora completos do usuário (811R4 + 12.1.2510, ~30.000 arquivos): aparecem em **0 a 1 arquivo no total** (`FWWizardControl` em 1 arquivo do 12.1.2510; as outras 7, zero). Além disso, nenhuma delas tem hoje um jeito de ser populada a partir de AdvPL real — não existe `TButton():New()`, `TabPage():New()` etc. construível em AdvPL; existem só como structs Go internos (`pkg/mvc/view.go`) nunca conectados ao VM/bytecode. Renderizá-las

de verdade exigiria inventar uma API de construção de widgets do zero, sem nenhum uso real nos corpora para validar contra. Apresentados os números, o usuário optou por não investir nisso agora — permanecem como estruturas de dados Go sem renderer (nem web, nem desktop), documentado aqui como decisão consciente, não lacuna esquecida.

Sem regressão em todo o arco: `go build ./...`, `go vet ./...`, `go test ./...`, `make test` (30 fixtures), sweep completo do corpus de 500 arquivos (500/500) a cada passo.

[1.10.0] — 2026-07-11

Banco de dados local automático — `RetSqlName/DbSelectArea/GetArea` funcionam sem dicionário configurado

Pedido do usuário: funções de acesso a tabela (`RetSqlName`, `DbSelectArea`, ...) deveriam funcionar independente de existir um dicionário de dados configurado, e cada diretório de trabalho deveria ganhar seu próprio banco SQLite automaticamente — sem exigir configuração prévia via `advcfg` — de forma que `advcfg` / `adveditor` rodados no MESMO diretório logo depois já enxerguem esse banco e permitam criar tabelas/campos/índices nele.

- **ResolveDatabasePath** (`pkg/tools/shared`, usado por `advplc/advcfg/adveditor/advpp-ide`): quando não há caminho explícito, variável de ambiente `ADVPP_DB`, nem um `~/.advpp/advpp_config.json` que REALMENTE exista em disco (antes o valor sintético que `LoadConfig` sempre devolve mascarava "nada configurado" como se fosse o banco global já escolhido), o padrão agora é um banco LOCAL `./advpp.db` no diretório de trabalho atual — não mais o global `~/.advpp/ADVPP.db`. O banco global só volta a valer depois que o usuário configura explicitamente via `advcfg`. Removidos os candidatos legados `./data/ADVPP.db` / `./data/advpl_dictionary.db` (simplificação).
- **OpenSQLite** (ponto único de abertura, `pkg/tools/shared`): agora cria um arquivo vazio ANTES de abrir quando o caminho não existe — `sql.Open` + `Ping` sozinhos não garantiam que o arquivo aparecesse em disco imediatamente (o driver só materializa no primeiro INSERT/CREATE real, que podia nunca acontecer se a primeira operação fosse uma leitura que falha por tabela inexistente). Sem isso, `advcfg` / `adveditor` rodados logo em seguida no mesmo diretório não viam banco nenhum para abrir.
- **attachDatabase** (`cmd/advplc`): removido o `os.Stat` que pulava a conexão inteira quando o arquivo ainda não existia — o VM agora sempre anexa um banco (criado na hora se preciso), em vez de rodar sem nenhum quando nada foi configurado.
- **Novas nativas**: `RetSqlName(alias)` devolve o próprio alias em maiúsculas (sem um dicionário SX2 carregado, é assim que as tabelas locais deste VM são nomeadas — funciona mesmo sem dicionário nenhum, como pedido). `GetArea()` / `RestArea(alias)` salvam/restauram a área de trabalho atual. `Alias()` (antes sempre `" "`) agora devolve a área atual de verdade.

`pkg/tools/shared` ganhou testes unitários pela primeira vez (isolados via `HOME` / diretório de trabalho temporários — nunca tocam a config real do usuário). Sem regressão: `go test ./...`, 30 fixtures, sweep completo do corpus de 500 arquivos.

[1.10.1] — 2026-07-11

Exclusão lógica (`recno/D_E_L_E_T_/R_E_C_D_E_L_`) e consolidação `advcfg` → `adveditor`

Pedido do usuário: adotar o padrão clássico Protheus de exclusão lógica nas tabelas gerenciadas pelo compilador/ferramentas, e descontinuar o `advcfg` por ser funcionalmente redundante com o `adveditor` — mantendo só uma ferramenta gráfica de banco de dados, evoluída para cobrir tudo que o `advcfg` fazia (criar/editar/excluir tabelas, campos e índices), com o `adveditor` como ponto único de evolução futura (ex.: outros bancos além de SQLite).

- **Exclusão lógica** (`pkg/tools/shared/database.go`): toda tabela criada via `CreateTable` ganha 3 colunas de sistema automáticas — `R_E_C_N_O_` (INTEGER PRIMARY KEY AUTOINCREMENT), `D_E_L_E_T_` (' ' / '*') e `R_E_C_D_E_L_` (0/1, espelho booleano para filtro SQL nativo). Leituras (`GetData` / `GetRecord` / `Count` / `Sum`) filtram `R_E_C_D_E_L_ = 0` por padrão; `DeleteRecord` passou a fazer UPDATE (marca como excluído) em vez de DELETE físico; `RecallRecord` reverte a exclusão; `Pack` purga fisicamente os registros marcados e roda VACUUM. Colunas de sistema ficam ocultas em `loadStructure()` e nomes de coluna são validados contra SQL injection (`validIdentifier`) antes de entrar na query.
- **cmd/advcfg removido inteiramente** (402 linhas) — assim como `pkg/tools/shared/dictionary.go` (579 linhas, abstração SX2/SX3/SIX específica do advcfg, sem uso fora dele) e `docs/MANUAL_ADVCFG.md`. `Makefile` e o workflow de release não compilam/empacotam mais o binário.
- **cmd/adveditor ganhou os métodos que faltavam**: CRUD completo de registro (Incluir/Alterar/Excluir com diálogos de formulário), e um novo menu "Tabela" com Nova Tabela (editor de campos dinâmico), Excluir Tabela, Adicionar/Remover Coluna, Criar/Excluir Índice — tudo com diálogos reais em vez de stubs.
- **3 bugs reais encontrados por teste visual** (Xvfb + xdotool + screenshot, não apenas `go build` / `go test`): a árvore de navegação nunca havia renderizado nenhum dado nesta ferramenta — faltava `ExtendBaseWidget` no widget customizado, e o código tratava a raiz da árvore como um ID fixo em vez do ID vazio convencionado pelo framework de UI; a grade de dados nunca era recarregada depois de Incluir/Alterar/Excluir porque a rotina de reabrir tabela detectava "já aberta" e devolvia dados obsoletos em vez de recarregar; o diálogo de "ver estrutura" renderizava como uma fresta de ~1px por falta de `Resize()` explícito no contêiner de rolagem, e perdia a formatação porque reprocessava texto já convertido como se ainda fosse markdown.

Sem regressão: `go build ./...`, `go vet ./...`, `go test ./...`, `make test` (30 fixtures), sweep completo do corpus de 500 arquivos (500/500).

advpp-ide: syntax highlight, banco auto-provisionado e integração com AdvEditor

Pedido do usuário: fazer o advpp-ide acompanhar a evolução do adveditor — não só visualmente, mas também com syntax highlight, as mesmas características de acesso/criação de banco de dados, e permitir abrir o AdvEditor a partir do próprio IDE. Também pedido: auditar se o advpp-ide reflete todas as evoluções do compilador até aqui.

- **Syntax highlight real** (`pkg/ui/editor.go`, `CodeEditor`): editor colorido para AdvPL/TLPP (palavras-chave, tipos, strings, comentários, números, diretivas `#`). `widget.Entry` do Fyne não suporta cor por token, então o editor alterna entre um `widget.Entry` nativo (100% do comportamento de edição — cursor, seleção, clipboard, IME) enquanto tem foco, e um preview colorido (`widget.RichText`) somente-leitura assim que perde o foco ou um arquivo é aberto; um clique no preview volta ao modo

de edição. As listas `advplKeywords` / `advplTypes` já existiam no arquivo mas nunca tinham sido usadas — a UI mostrava um `widget.Entry` puro sem nenhuma coloração.

- **3 bugs reais de renderização Fyne encontrados via teste visual**

(Xvfb+xdotool+screenshot, invisíveis a `go build` / `go test`) ao construir

o alterna-preview: (1) `widget.RichText` só resolve seu

`BaseWidget.impl` preguiçosamente, no primeiro `CreateRenderer()` /

`MinSize()` — se o widget é escondido antes da primeira pintura, esse

gatilho nunca ocorre e todo `Refresh()` / `Show()` posterior vira no-op

silencioso para sempre (corrigido chamando `MinSize()` uma vez no

construtor). (2) `Container.Refresh()` (Fyne v2.4.4) nunca chama

`SetDirty()` no canvas — só `Container.Move()` / `Hide()` chamam — então

trocar `Objects` de um `container.Stack` em runtime e chamar só

`Refresh()` não repinta a tela (corrigido forçando um `Move()` para a

própria posição atual logo depois). (3) o bug real, mais sutil: o

widget "catcher" transparente usado para capturar o clique que volta ao

modo edição estava pintando um retângulo *opaco* da cor de fundo por

cima do preview colorido (`container.Stack` pinta objetos depois por

cima dos anteriores) — cobria o próprio texto que deveria deixar

visível. Corrigido trocando para `color.Transparent`.

- **Banco de dados auto-provisionado:** `run()` usava o padrão antigo

(`os.Stat` + `SetDBEngine` condicional, pulando a conexão se o arquivo

ainda não existisse) — trocado pelo mesmo padrão `SetDBFactory` de

`attachDatabase` já usado em advplc desde a v1.10.0 anterior, então

`RetSqlName` / `DbSelectArea` funcionam a partir do advpp-ide sem

configuração prévia, com o mesmo banco local `./advpp.db`.

- **Menu Tools → "Open AdvEditor (database)":** lança o binário `adveditor`

como processo separado, procurando primeiro ao lado do executável do

advpp-ide (mesmo layout dos pacotes de release) e depois no PATH.

- **Versão real no About/título:** `advplc` / `adveditor` / `advpp-ide` já

recebiam `-X main.version=` via `make release`, mas só `advplc`

declarava a variável `version` — nos outros dois o link `-X` era um

no-op silencioso e a UI sempre mostrava texto hardcoded ("v1.0",

"Version 1.0.0"). Corrigido nos dois.

- **Auditoria de paridade com o compilador:** advpp-ide importa

`pkg/compiler` / `pkg/vm` / `pkg/parser` diretamente do mesmo módulo, então

motor LLM, MCP e `#command` já chegavam automaticamente — o único gap

real era o padrão de banco de dados acima.

Sem regressão: `go build ./...`, `go vet ./...`, `go test ./...` (incluindo testes novos de `pkg/ui` para o tokenizer de highlight), `make test` (30 fixtures), sweep completo do corpus de 500 arquivos (500/500).

Fix: testes de `ResolveDatabasePath` falhavam no CI do Windows

`config_test.go` comparava o resultado de `ResolveDatabasePath` contra

strings absolutas estilo Unix hardcoded (`/custom/path.db`), mas a função

sempre normaliza via `filepath.Abs`, que no Windows reescreve esse literal

como `C:\custom\path.db` (ancorado na unidade atual) em vez de preservar a

string literal — e o teste que checava a prioridade do config real em disco

usava um caminho que `filepath.IsAbs` nem reconhece como absoluto no

Windows (sem letra de unidade), então o próprio código de produção caía no

fallback local em vez de exercitar o caminho testado. Corrigido calculando

o valor esperado com o mesmo `filepath.Abs` (não mais um literal Unix) e

usando um caminho genuinamente absoluto (`t.TempDir()`) no teste do config.

Confirmado verde nos 3 SOs via GitHub Actions após o fix.

[1.9.1] — 2026-07-11

4 bugs reais de parser encontrados em validação fora do corpus de amostra

- `SET DATE FORMAT "dd/mm/yyyy"` (Clipper clássico sem `T0`) não era reconhecido pelo dispatcher de `SET`, que só cobria `SET <opção> T0/ON/OFF/OF ...`.
- Nome de variável colidindo (case-insensitive) com uma keyword de tipo TLPP (`Date`, `Array`, `Object`, ...) era rejeitado em posição de declaração (`Local` / `Private` / `Public` / `Static`), que exigia `TOKEN_IDENT` estrito em vez de aceitar keyword também — já aceito em nome de parâmetro de função, agora consistente também em declaração.
- Keyword-colisão como valor puro no fim de expressão (ex.: `Return <nomeQueColideComKeyword>`), sem token de continuação depois na mesma linha para o heurístico existente reconhecer) — fallback final adicionado em `parsePrimary`: quando nenhuma outra forma bate, uma keyword em posição de operando vira identificador comum.
- `(ALIAS_MACRO)->campo := valor` onde `ALIAS_MACRO` é um `#define` para uma STRING (idioma real e comum para nomear alias de tabela temporária) — depois da expansão da macro, `alias->campo` só era reconhecido quando o lado esquerdo do `->` era um `*ast.Ident`; com string literal, o `->campo` era descartado silenciosamente e a atribuição sobrava com a STRING como alvo ("unsupported assignment target: *ast.StringLit"). Corrigido em `parsePostfix` para os dois lugares que constroem `FieldAccess` a partir do lado esquerdo de `->`.
- Mensagem de erro do codegen para alvo de atribuição não suportado agora inclui o número da linha (facilita bisseção de bugs reais).

Sem regressão: `go test ./...`, 30 fixtures, sweep completo do corpus de 500 arquivos.

[1.9.0] — 2026-07-11

Rodada de otimizações de performance (compilador, VM, motor de inferência LLM)

Todas as mudanças abaixo são puramente de performance — comportamento e saída idênticos, validados por toda a suíte de testes (incluindo `TestValidateAgainstLlamaCPP`, que compara geração greedy token a token contra o llama.cpp de referência) e pelo sweep completo do corpus Protheus (500/500, 100% mantido). Descobertas via `pprof` sobre um forward pass real (Falcon3-3B-1.58bit) e uma compilação real (arquivo Protheus de 3,6MB) — não especulação.

Motor de inferência LLM (`pkg/llm/`) — ~14x mais rápido por forward pass nesta máquina (5,27s → 0,36s):

- `Σq` (correção do caminho AVX2 de `MatMulI2S`) era recalculada em TODA linha do peso via `sumInt8`, embora seja o mesmo `q` (ativação quantizada) para as `NRows` linhas — ~32% do tempo total, hoisted para fora do loop de linhas (calculada uma vez por chamada de `MatMulI2S`).
- `Float16ToFloat32` trocou a decodificação bit-a-bit por uma tabela de 65536 entradas pré-computada em `init()` — usada por `MatMulF16` (a maior matmul do forward pass, projeção de saída com `vocab_size` linhas), que sozinha era ~34% do tempo antes desta troca. Validada exaustivamente contra a implementação de referência para os 65536 padrões de bit possíveis.
- Novo kernel `dotF16BlocksAVX2` (amd64, `simd_amd64.s`) usa F16C (`VCVTPH2PS`, conversão half → float em hardware) + FMA (`VFMADD231PS`) para `MatMulF16` — mesmo que a tabela já tivesse ajudado, essa matmul ainda dominava o profile. Detecção de CPU (`hasF16CFMA`) via CPUID, com fallback escalar (tabela) em CPUs sem F16C/FMA e em arquiteturas não-amd64.

- `gguf.go`: o parser do header GGUF (metadados + lista de tensores) fazia uma syscall `pread` por CAMPO (nome, cada dimensão, tipo, offset — de cada tensor, mais cada entrada de metadado) via `os.File.ReadAt` direto; para um modelo real (~150+ tensores) isso sozinho era mais tempo que ler os dados de peso em si. Trocado por `bufio.Reader` — `LoadModel` ficou ~2,2x mais rápido (2,75s → 1,26s).

Compilador (`cmd/advplc` , `pkg/preprocessor` , `pkg/lexer`) — ~1,9x mais rápido por arquivo grande (163ms → 85ms num arquivo Protheus real de 3,6MB):

- `processFile` (preprocessor) calculava `strings.ToUpper` da LINHA INTEIRA para TODA linha do arquivo, só para checar prefixos de diretiva (`#INCLUDE` , `#DEFINE` , ...); ~31% do tempo de compilação. Agora só computa quando a linha já começa com `#` ou `BeginSql`.
- `convertWithGoEncoding` (conversão CP-1252 → UTF-8) escrevia byte a byte num `bytes.Buffer` (~23-48% do tempo, dependendo da correção anterior já aplicada ou não); agora copia em BLOCOS as sequências contíguas de bytes ASCII (a esmagadora maioria de qualquer fonte real) com um único `Write` por trecho, só indo byte a byte nos raros bytes >=128 (acentos em comentários), via tabela pré-computada.
- `tryBeginContent` (lexer) fazia um `strings.EqualFold` de 12 bytes a cada identificador do arquivo inteiro para checar `BeginContent`; agora um `if` de 1 byte descarta a esmagadora maioria antes do `EqualFold`.

VM/interpretador (`pkg/runtime`) — ~28% mais rápido num loop aritmético sintético (2,65s → 1,90s para 2M iterações):

- `NewNumber` alocava um `*NumberValue` novo no heap a cada chamada — TODA operação aritmética do VM (soma, subtração, módulo, comparação) aloca um resultado novo, e isso dominava o profile de qualquer loop (`runtime.mallocgc` / `newobject` bem acima de qualquer opcode). Adicionado cache de inteiros pequenos (-256..4096, cobrindo contadores de loop, índices de array, resultados de módulo/comparação comuns) — `NewNumber` devolve o ponteiro compartilhado em vez de alocar, seguro porque `NumberValue` é imutável depois de criado e `Equals` compara por valor (mesmo padrão já usado pelo singleton `Nil`). `pkg/runtime` ganhou testes unitários pela primeira vez (`values_test.go`) para essa mudança.

Achado, não resolvido nesta rodada: a raiz do custo é `Value` ser uma interface (todo número é um ponteiro boxed no heap, nunca um valor inline) — resolver isso de verdade exigiria trocar a representação de `Value` para um tagged union/valor inline, uma mudança arquitetural que toca `pkg/vm` , `pkg/runtime` e `pkg/compiler` inteiros; fora do escopo seguro de uma rodada de perf num código que acabou de chegar a 100% de correteude. Documentado aqui para uma futura sessão dedicada.

Portabilidade multi-SO: nenhuma mudança usa `unsafe` , `cgo` , ou qualquer API específica de SO — `bufio` / `bytes.Buffer` / tabelas são 100% Go puro, idênticas em linux/darwin/windows. O kernel `dotF16BlocksAVX2` é amd64-only (gated por `hasF16CFMA` , detecção em runtime) com fallback escalar já testado em todas as arquiteturas; darwin/arm64 (Apple Silicon, o alvo `darwin-arm64` do release) e demais arquiteturas usam o caminho escalar + tabela, que já é ~6x mais rápido que antes desta rodada (as duas primeiras otimizações do LLM não dependem de amd64). Um kernel NEON para arm64 foi considerado mas não implementado nesta rodada: esta máquina de desenvolvimento não tem hardware nem emulação de usuário arm64 disponível (só QEMU full-system, sem `binfmt_misc` registrado; instalar `qemu-user-static` ou usar `docker run --privileged` para registrar emulação foram bloqueados pelo classificador de segurança do Auto Mode por serem mudanças de sistema não solicitadas explicitamente) — escrever

assembly SIMD sem conseguir RODAR os testes contra hardware real não é uma otimização segura de se enviar. Ver "Próximos passos" abaixo.

Próximos passos possíveis (não feitos nesta rodada, por escopo/risco):

kernel NEON arm64 para `dotI2SBlocksAVX2` / `dotF16BlocksAVX2` (precisa de CI rodando em runner macOS arm64 real para validar — o repo já tem `macos-latest` no `release.yml`, mas só builda, não roda `go test`; um workflow de CI dedicado a testes, multi-SO, seria o pré-requisito seguro); representação de `Value` sem boxing no VM (projeto arquitetural à parte).

[1.8.7] — 2026-07-11

Sweep de pass-rate no corpus Protheus real (98,6% → 100%, 500/500)

- **Bug estrutural (parser):** cláusulas soltas de DSL (`ADJUST` / `NO BORDER` / `OF` /...) depois de uma expression statement eram consumidas mesmo quando o próximo token estava em outra linha — o comentário do código já dizia "same line only", mas faltava o check de `.Line` de fato. `isAtClauseWord` reconhece palavras genéricas como `ITEM`, `SIZE`, `TO`, `OF`, `COLOR`, `FONT` — nomes comuns de variável/campo — então um statement solto (`item`) seguido na linha seguinte por `item := {}` (ITEM é cláusula de RADIO/COMBOBOX) comia o segundo `item` como cláusula e sobrava um `:=` "inesperado". Agora exige `p.tokens[p.pos-1].Line == p.peek().Line` para entrar/continuar no loop.
- **Bug estrutural (lexer + cmd/advplc):** `cmd/advplc` converte fontes CP-1252 para UTF-8 antes de lexar; um NBSP (0xA0) colado por editores de texto vira a sequência UTF-8 de 2 bytes `0xC2 0xA0`, não um `0xA0` solto. A heurística do lexer "byte >= 0x80 é letra acentuada CP-1252" engolia o `0xC2` para dentro do identificador seguinte (`MSGGET\xc2\xa0cBanco` virava um único identificador corrompido), desalinhando a tokenização e quebrando o parse 1-2 statements depois — o clássico "drift bug" da sessão. Faixa de tokens idêntica entre a versão corrompida e a versão limpa em ferramentas de depuração que leem bytes crus (sem a conversão CP1252 → UTF8) mascarou o diagnóstico por várias iterações. Corrigido reconhecendo a sequência de 2 bytes como espaço em branco tanto em `skipWhitespace` quanto no scanner de identificadores (`tokenizeIdentifier` para de consumir ao encontrá-la).
- Pass-rate final: **500/500 (100%)** no corpus de amostra (811R4 + 12.1.2510).

[1.8.6] — 2026-07-10

Sweep de pass-rate no corpus Protheus real (96,8% → 98,6%)

- `WEB EXTENDED INIT <var> START <expr>` / `WEB EXTENDED END` (Portais).
- `TTALK "v1"` como cláusula de WSMETHOD (declaração e dispatch); `PRODUCES/CONSUMES` aceita constante que colide com keyword; `QUERYPARAM a, b` (lista) na implementação.
- `Replace <campo> With <expr> [, <campo> With <expr>...]` (Clipper).
- Flags/cláusulas de `@`: `CENTERED`, `RAISED`, `PROMPTS` (FOLDER), `FILENAME/FILE/DISK` (BITMAP).
- `COLORS` como sinônimo de `COLOR` no `DEFINE`.
- `Store Header/Cols ... TO ...` tolerado nativamente quando o `.ch` com o `#command` (PLSMGER.CH) não é resolvível.

[1.8.5] — 2026-07-10

Sweep de pass-rate no corpus Protheus real (95,8% → 96,8%)

- **Bug estrutural (preprocessor):** diretivas `#IFDEF / #ENDIF` DENTRO de comentário de bloco (`/*#IFDEF TOP ... #ENDIF*/`, idioma real de desativar código) eram consumidas como diretivas de verdade, DELETANDO o `*/` da saída e invertendo a fase de comentários do resto do arquivo — fonte do drift bug mais antigo da sessão (WSAPD010). O preprocessor agora rastreia estado de comentário de bloco e passa essas linhas intactas.
- `Store <expr> To <var,...>` (atribuição múltipla Clipper).
- `DEFINE TIMER ... INTERVAL <n> ACTION <expr> OF <janela>`.
- Nome de parâmetro de função pode colidir com keyword (`Static Function GetSU5(oApi, Self)`).
- Vírgula pendurada antes de cláusula em lista de valores (`HEADER "a","b",; + SIZE w,h` na linha seguinte).

[1.8.4] — 2026-07-10

Sweep de pass-rate no corpus Protheus real (94,4% → 95,8%)

- **Bug estrutural:** `Static cVar := valor` DENTRO de uma função era tratado como fronteira de função (por causa de `Static Function`), truncando o corpo silenciosamente e corrompendo o parse do resto do arquivo (fonte dos piores "drift bugs"). STATIC agora só é boundary seguido de FUNCTION.
- DSL XML legado: `ADDNODE <expr> NODE <expr> ON <expr>`, `DELETENODE <expr> ON <expr>`, `CREATE <var> XMLFILE <expr> [SETASARRAY <lista>]`.
- Alvo de `Count/Sum/Average ... To` pode ser expressão (`self:nTotReg`).
- Keyword como identificador comum em posição de operando quando o token seguinte só continua expressão (`{|Panel| f(Panel, ...)}`).
- `Private &("nome"+var) := x` — memvar com nome computado por macro.
- `HEADERS` (plural) como cláusula de LISTBOX; WSMETHOD REST sem nome próprio (`WSMETHOD GET WSRECEIVE ... WSSERVICE X`).

[1.8.3] — 2026-07-10

Sweep de pass-rate no corpus Protheus real (92,6% → 94,4%)

- `@ nLin++` sozinho (forma degenerada real) tolerado como expressão.
- `End If / End Do` fechando If e Do Case (variantes de duas palavras).
- QUALQUER keyword seguida de `(` na mesma linha em contexto de expressão é chamada de função (`alias->(Add())`, `Select()`, ...) — generalização da lista IF/ARRAY/DATE/OBJECT/BREAK.
- Nome de função pode colidir com keyword (`Static Function Add`).
- `SEND MAIL FROM ... TO ... SUBJECT ... BODY ... [ATTACHMENT] [RESULT]` e `GET MAIL ERROR <var>` (DSL de e-mail do workflow).
- `MENU <var> POPUP ... MENUITEM ... ACTION ... ENDMENU` (menu de contexto).
- `DEFINE DBTREE ... CARGO ; ON CHANGE <expr>`; `DBADDTREE ... PROMPT ... RESOURCE ... CARGO ... OPENED` (árvores legadas).
- Cláusulas de `@`: `FROM` (METER FROM 0 TO 100), `PICT` (abrev. de PICTURE), `OPTION` (FOLDER), flag `RIGHT`; `FIELDS` pode levar lista de valores própria (`LISTBOX ... FIELDS "" ; HEADER ...`).

[1.8.2] — 2026-07-10

Sweep de pass-rate no corpus Protheus real (89,4% → 92,6%)

- `&&` (comentário Clipper) após `;` de continuação tratado como fim de linha para a continuação (lexer), igual ao `//` já suportado.
- `Begin Report Query <expr>` — a seção pode ser expressão completa (`oReport:Section(2)`), não só um nome, nos dois lados do bloco.
- Atribuição encadeada como valor dentro de item de codeblock (`x[9] := x[10] := ... := 0`).
- `COLORS` como cláusula de `@` (FOLDER ... COLORS 0,167...).
- `Release Object <nome>` / `Release All [Like <máscara>]`.
- `Break(oErro)` como chamada de função em expressão (idioma de ErrorBlock), distinto do statement BREAK.
- `@ y,x To y2,x2 MultiLine Object oMulti` — flags/cláusulas do TMultiGet legado no ramo de caixa do `@`.
- `Data <keyword>` — nome de membro de classe pode colidir com palavra reservada (`Data size`, `Data default`); `::Default()` idem em acesso.
- `DEFINE SCROLLBAR ::oVScroll VERTICAL OF Self RANGE a,b` — alvo `::prop` em DEFINE e ACTIVATE, flags VERTICAL/HORIZONTAL e cláusula RANGE.
- Bytes de controle soltos (`\x01`, corrupção comum em fontes legados) tolerados pelo lexer, como o backtick.
- `Count To <var>` / `Sum <exprs> To <vars>` / `Average ... To ...` (comandos Clipper de agregação; parseados e descartados).
- Atribuição em resultado de chamada (`ATail(arr) := v`), semântica de referência do Clipper) tolerada no codegen (avalia e descarta), mesma tolerância já dada a `¯o := v`.

[1.8.1] — 2026-07-10

Sweep de pass-rate no corpus Protheus real (87,6% → 89,4%)

- Resolução de `#include`: tenta subpastas convencionais (`ch/`, `include/`, `includes/`) e fallback case-insensitive por diretório (fontes CP-1252 vindos de Windows quase nunca batem o case do disco em Linux).
- Junção de linha lógica antes do casamento de `#command` (`Store COLS ... ;`)
 - `While ...`), preservando contagem de linhas; comentário `//` após o `;` de continuação não esconde mais a marca (definições e uso); CRLF (`;\r`) tratado em todos os joins.
- Operador de exponenciação `^` e sinônimo `**` (parsePower, associativo à direita, acima da multiplicação; novo OP_POW na VM via math.Pow).
- `ACTIVATE FWMBROWSE/MBROWSE/REPORT <var>`.
- DSL mobile FDA: `ADD FOLDER ... CAPTION ... [ON ACTIVATE f()] OF ...`, `ADD COLUMN ... TO ... ARRAY ELEMENT n HEADER ... WIDTH n`, `SET BROWSE <var> ARRAY <expr>`, cláusulas `CAPTION` / `SYMBOL` / `ITEM` / `VSCROLL` e flags de duas palavras `NO SCROLL` / `NO UNDERLINE` em `@`.

[1.8.0] — 2026-07-10

Motor de #command reescrito para semântica Clipper real + 2 features de lexer (86,4% → 87,6%)

Rodada dirigida pelos arquivos que exigiam o pré-processador de verdade:

Motor #command/#xcommand (pkg/preprocessor):

- Junção de definição multi-linha agora REMOVE o `;` de continuação de fim de linha (antes ficava dentro do padrão como literal impossível de casar, desativando silenciosamente toda regra multi-linha); um `;` no meio/começo de linha do resultado é conteúdo (separador de comandos gerados).
- Regra definida por ÚLTIMO vence (ordem reversa de tentativa), semântica

Clipper que permite um .ch especializar comando já definido.

- Cláusulas opcionais consecutivas casam em QUALQUER ORDEM (ex.: padrão declara `[OF <oWnd>] [PIXEL]`, fonte usa `PIXEL OF oPanel`).
- Marcador restrito multi-literal `<nome: LIT1, LIT2, ...>` (captura qual literal casou; `<.nome.>` vira .T./F.).
- Marcador guloso para em QUALQUER literal alcançável à frente (união dos abridores de todos os grupos opcionais + próximo literal obrigatório), não só no primeiro.
- Marcador dentro de grupo opcional herda o ponto de parada do padrão externo (lista, não um único literal).
- Grupos opcionais `[...]` do lado do RESULTADO: emitidos sem os colchetes só se algum marcador interno capturou algo (antes viravam `[, ]` literal).
- Marcador de resultado `<"var">` (stringify: nome capturado entre aspas).
- Expansão re-processada recursivamente (o resultado pode conter novos comandos, ex.: VTSAY+VTGET expande para dois comandos `@...` que expandem de novo), segmentada por `;` de topo.
- Comentário `//` de fim de linha removido antes do casamento (um marcador guloso o capturava para dentro da expansão, comentando o resto do código gerado).
- Flag `[<nome: LITERAL>]` com espaços ao redor de `:` `/` `,` normalizada.

Lexer:

- Literal de string entre colchetes do Clipper: `[texto]` / `[]` em posição de operando é string (heurística clássica: `[` após token que encerra operando é indexação; caso contrário, literal até o `]` da mesma linha).
- `BeginContent var <nome> ...conteúdo cru... EndContent` (bloco TLPP de JSON/XML embutido) — corpo não é AdvPL e não deve ser tokenizado; consumido no lexer e emitido como `<nome> := "<corpo>"`.

Parser:

- Parâmetro de codeblock pode colidir com palavra reservada `{|Self| ...}` — aceita keyword como nome.

[1.7.11] — 2026-07-10

Sweep de pass-rate no corpus Protheus real (86,0% → 86,4%)

Continuação do sweep dirigido por corpus (ver `[[advpp_corpus_locations]]`).

Dois bugs reais adicionais de parser corrigidos:

- `WSMETHOD GET <nome> PATHPARAM <param> WSRECEIVE ...` — cláusulas `REST PATHPARAM / QUERYPARAM` (binding de parâmetro de rota) não reconhecidas na implementação do `WSMETHOD`.
- `Default Self:Prop := valor` — alvo de `Default` explicitamente escrito como `Self:Prop` em vez do atalho `::Prop` já suportado.

[1.7.10] — 2026-07-10

Sweep de pass-rate no corpus Protheus real

- `@ y,x BMPBUTTON TYPE n ACTION expr` — cláusula `TYPE` (número de estilo do botão bitmap) não reconhecida no laço de cláusulas de `@`.

[1.7.9] — 2026-07-10

Sweep de pass-rate no corpus Protheus real (85,2% → 86,0%)

Continuação do sweep dirigido por corpus (ver [[advpp_corpus_locations]]).

- **Bug estrutural (lexer):** literal numérico com ponto decimal sem dígito antes (`.5`, `.7`) — comum em coordenadas (`@.5,.7...`) não era reconhecido; o lexer só entrava no tokenizador de número ao ver um dígito primeiro, então um `.` inicial caía sempre no caminho de dot-literal/operador (`.T.`, `.AND.`, `.`), nunca no de número. Corrigido: um `.` seguido de dígito agora entra no tokenizador de número.

[1.7.8] — 2026-07-10

Sweep de pass-rate no corpus Protheus real (84,4% → 85,2%)

Continuação do sweep dirigido por corpus (ver [[advpp_corpus_locations]]).

Dois bugs reais adicionais de parser corrigidos:

- `DEFINE CELL ... BLOCK{||...} ...` — cláusula de bloco de valor da coluna do TReport não reconhecida.
- `@ x1,y1 TO x2,y2 DIALOG <var> TITLE "... " [PIXEL]` — sintaxe legada de criação de diálogo via `@ ... TO ...` (equivalente a `DEFINE MSDIALOG <var> FROM x1,y1 TO x2,y2 TITLE ...`), confundida com o desenho de caixa (`@ ... TO ... BOX`); nenhum dos dois é o outro, precisavam de ramos separados.

[1.7.7] — 2026-07-10

Sweep de pass-rate no corpus Protheus real (83,6% → 84,4%)

Continuação do sweep dirigido por corpus (ver [[advpp_corpus_locations]]).

Dois bugs reais adicionais de parser corrigidos:

- `Private M->NOME_CAMPO := valor` — idioma Clipper de qualificar explicitamente uma memvar com o alias "M" (memory), redundante mas usado em fontes reais para dar nome de memvar igual a um campo; `Local` / `Private` / `Public` / `Static` só aceitavam um nome simples, não o padrão `M->nome`.
- `@ ... MSGET ... HASBUTTON F3 "... " ...`, `@ ... WORKTIME ... RESOLUTION <expr> VALUE <expr> ...` — cláusulas de `@` não reconhecidas (`HASBUTTON` do MSGET com botão de F3; `RESOLUTION` / `VALUE` do controle WORKTIME).

[1.7.6] — 2026-07-10

Sweep de pass-rate no corpus Protheus real (83,0% → 83,6%)

Continuação do sweep dirigido por corpus (ver [[advpp_corpus_locations]]).

- `alias->END` (e qualquer outro campo cujo nome colide com palavra reservada, ex. `alias->DELETE`) — nome de campo após `->` exigia `TOKEN_IDENT`; agora usa `expectName()` (aceita `TOKEN_KEYWORD` também), mesma classe de bug já corrigida em outros pontos do parser para identificadores que colidem com keywords.

[1.7.5] — 2026-07-10

Sweep de pass-rate no corpus Protheus real (82,0% → 83,0%)

Continuação do sweep dirigido por corpus (ver [[advpp_corpus_locations]]).

Dois bugs reais adicionais de parser corrigidos:

- Elemento de array literal `{a, b := c, d}` (sem `||`, usado como sequência de expressões em cláusulas VALID/ACTION reais) não aceitava atribuição (`:=`) como elemento — usava `parseExpression` puro em vez de `parseCodeBlockItem`, mesma classe de bug já corrigida em outros pontos do parser para `:=` inline.
- `Return target := value` (atribuição usada inline como valor de retorno, ex. `Return self.oProp := {...}`) deixava o `:=` pendurado — corrigido nos DOIS parsers de RETURN existentes (`expressions.go` e o parser de corpo de método em `parser.go`, que tem sua própria cópia da lógica de RETURN) para usar `parseAssignRHS` em vez de `parseExpression`.

[1.7.4] — 2026-07-10

Sweep de pass-rate no corpus Protheus real (81,0% → 82,0%)

Continuação do sweep dirigido por corpus (ver `[[advpp_corpus_locations]]`).

Seis bugs reais adicionais de parser corrigidos:

- `@ ... RADIO/CHECKBOX ... 3D SIZE w,h ...` — flag de layout "3D" (tokeniza como NUMBER "3" + IDENT "D") não reconhecida no laço de cláusulas de `@` (só existia o caso equivalente em `DEFINE`).
- `LOCATE FOR <expr> [WHILE <expr>]` — comando Clipper de busca sequencial no alias atual, não suportado (nenhum dispatch existia).
- `Copy File <expr> To <expr>` — cópia de arquivo em disco (comando Clipper), distinto de `Copy To` (exportação de registros); confundia-se com este e quebrava o parsing.
- `Copy <alias-expr> To Memory <name> [Blank]` — copia a estrutura de campos de um alias para um array; forma de `COPY` não reconhecida.
- `@ ... GET ... MULTILINE ... HSCROLL ...` — cláusulas do GET multilinha (TGet memo) não reconhecidas.
- `DEFINE SBUTTON ... ONSTOP <expr> ...` — cláusula de tooltip do botão não reconhecida.

[1.7.3] — 2026-07-10

Sweep de pass-rate no corpus Protheus real (76,6% → 81,0%)

Continuação do sweep dirigido por corpus (ver `[[advpp_corpus_locations]]`).

Dois bugs estruturais de alto impacto e mais quatro bugs pontuais de parser corrigidos:

- **Bug estrutural:** strings `"..."` sem aspa de fechamento até o fim da linha (typo comum em fontes reais, ex. query SQL multi-linha via `+=`) travavam o lexer, que consumia até a próxima aspa em QUALQUER linha seguinte, engolindo o resto do arquivo. Clipper/AdvPL fecha strings implicitamente no fim da linha; o lexer agora faz o mesmo.
- **Bug estrutural:** um identificador seguido de `(` seguinte, sem guarda de mesma linha, era sempre tratado como chamada de função — já que newlines são removidas antes do parsing, um `(alias)->campo` no início da PRÓXIMA linha grudava como argumento da chamada do identificador da linha anterior (`var := f() \n (alias)->campo := x` virava `f()(alias)->campo`). Corrigido em dois pontos: `parsePrimary` (chamada direta `ident()`) e `parsePostfix` (chamada após expressão composta), ambos agora exigem que o `(` esteja na mesma linha do token anterior.
- `alias->(expr1, expr2, ...)` — sequência separada por vírgula dentro do escopo de alias só aceitava uma única expressão; agora usa a mesma produção de `(a, b, c)` (avalia todas, retorna a última).

- `DEFINE SECTION ... TABLES "A","B",...`, `DEFINE CELL ... PICTURE "..."`, `DEFINE BREAK ... WHEN {||...}`, `DEFINE FUNCTION ... FUNCTION SUM BREAK oBreak TITLE "..." NO END SECTION` — cláusulas do DSL de TReport
(`DEFINE SECTION/CELL/BREAK/FUNCTION`) não reconhecidas: `TABLES`, `PICTURE`, `WHEN`, `FUNCTION` (como nome de cláusula, colide com o nome do próprio `DEFINE` kind), `BREAK`, e o flag de três palavras `NO END SECTION`.

[1.7.2] — 2026-07-10

Sweep de pass-rate no corpus Protheus real (73,8% → 76,6%)

Continuação do sweep dirigido por corpus (ver [[advpp_corpus_locations]]).

Seis bugs reais adicionais de parser corrigidos:

- `If x := cond` / `While x := cond` — atribuição usada inline como condição (idioma comum em AdvPL: "avança e testa") deixava o `:=` pendurado; a condição agora usa `parseAssignRHS` como o resto do parser.
- Caminho de namespace TLPP totalmente qualificado
(`totvs.framework.treports.date.stringToTimeStamp(...)`) quebrava sempre que um segmento colidia com palavra reservada (`date`, que lexava como `TOKEN_KEYWORD`); o loop de segmentos só aceitava `TOKEN_IDENT`. Mesmo problema corrigido em `NAMESPACE / USING NAMESPACE`, agora via `parseNamespacePath` compartilhado (segmento só aceito logo após um ponto, nunca solto — não avança para além da declaração).
- `WSRESTFUL/WSSERVICE <nome> ... FORMAT <expr>` — cláusula de cabeçalho não reconhecida, quebrando o corpo inteiro do bloco.
- Bug de colisão de nome em `WSDATA`: o bypass "nome de método é opcional" (`WSMETHOD POST DESCRIPTION "..." ...`) se aplicava também a `WSDATA`, então um campo literalmente chamado `Description` (`WSDATA Description As String`) era confundido com a cláusula `DESCRIPTION` e o parser pulava o nome do campo — struct inteira corrompida a partir daí. `WSDATA` agora sempre exige nome explícito.
- `ParamType <n> Var <nome> As <tipo> [Default <expr>]` — declaração de metadados de parâmetro não suportada.

[1.7.1] — 2026-07-10

Sweep de pass-rate no corpus Protheus real (70,8% → 73,8%)

Continuação do sweep dirigido por corpus contra os fontes reais 811R4 e

12.1.2510 (amostra de 500 arquivos, ver [[advpp_corpus_locations]]).

Onze bugs reais de parser corrigidos:

- `SET KEY <nKey> TO [<uBlock>]` — o keycode antes do `TO` não era reconhecido pelo dispatcher de `SET`.
- `DEFINE CELL ... AUTO SIZE` (TReport) — flag `AUTO` sem valor antes de `SIZE` não era reconhecida, quebrando o parsing da cláusula seguinte.
- Drift em `SET FILTER TO` — a heurística "tem valor?" não detectava que um `x += ...` na linha seguinte não era o valor do `SET`, engolindo a variável errada.
- `DELETE FILE <expr>` e `DELETE [FOR/WHILE/RECORD/REST/ALL]` — comandos Clipper de arquivo/registo não eram suportados.
- `WSMETHOD ... WSRECEIVE a,b WSEND c` — só aceitava `WSEND` antes de `WSRECEIVE` e com valor único; real Protheus usa qualquer ordem e listas separadas por vírgula em ambas.
- `PREPARE ENVIRONMENT EMPRESA/FILIAL/MODULO/TABLES` — comando batch de abertura de ambiente não suportado.

- `SET DELETE ON` — "DELETE" é palavra reservada (`TOKEN_KEYWORD`), não identificador; o dispatcher de `SET` exigia `TOKEN_IDENT` e falhava.
- **Bug estrutural:** como quebras de linha são descartadas antes do parsing, um `++x` prefixo iniciando uma nova instrução colava-se ao fim da expressão da instrução anterior (`y := f()` seguido de `++x` virava `(f())++`, erro de compilação "unsupported assignment target"). Corrigido exigindo que o operador pós-fixado `++` / `--` esteja na mesma linha do token anterior.
- Atribuição encadeada (`a:=b:=c:=valor`) dentro de `Local` / `Private` não era suportada (só funcionava em atribuição solta).
- `For ... EndFor` (além de `Next` / `End`) não fechava o loop.
- `Default a:=1, b:=2, c:=3` (múltiplas variáveis separadas por vírgula) só suportava uma única variável.

Motor real de `#xcommand` / `#command` / `#xtranslate` / `#translate`

Até aqui, o pré-processador **reconhecia** a sintaxe destas diretivas mas **descartava** as definições — nenhuma expansão de verdade acontecia. Isso quebrava qualquer arquivo que dependesse de comandos customizados definidos

em headers `.ch` reais (padrão comum em código Protheus legado: `STORE HEADER <CA> TO <aH> [FOR <for>]`, `COPY <cAC> TO MEMORY [<bl:BLANK>]`,

etc.). Agora o AdvPP implementa o pattern-matching de verdade, no estilo Clipper:

- **Padrão de casamento:** palavras literais (case-insensitive), `<nome>` (captura uma cláusula até o próximo literal esperado), `<nome,...>` (captura uma lista), `[...]` (grupo opcional — só tenta se o primeiro literal dele aparecer na posição atual), `[<nome:LITERAL>]` (marcador de flag booleana).
- **Molde de resultado:** `<nome>` (substitui pelo texto capturado, ou vazio se ausente), `<{nome}>` (vira `{|| texto}` se presente, `NIL` se ausente — usado para condições `FOR` / `WHILE` que viram codeblock), `<.nome.>` (`.T.` / `.F.` conforme presença), `\[ / \]` (colchete literal).
- Definições multi-linha via continuação com `;` (convenção usual do Clipper) são unidas antes de compilar a regra.

Três bugs reais adicionais encontrados e corrigidos no caminho (achados ao validar contra headers `.ch` reais de um fork ApSoft/Protheus):

1. **#define com múltiplos espaços** (`#define NOME valor`) — `parseDefine` usava `strings.SplitN(line, " ", 3)`, que quebra quando há mais de um espaço entre `#define` e o nome (comum em código real), armazenando a macro com nome vazio.
2. **#define multi-linha** (`#define NOME { "a","b",;\n "c","d" }`) — sem juntar as linhas de continuação, o resto do array vazava como código bruto (token solto no meio de uma statement).
3. **Tokenização por espaço simples** grudava identificador com pontuação colada (`TCSQLEXEC("select 1")` virava um token só), fazendo até um `#translate` sem parâmetros nunca casar; e quando um padrão casava só o início da linha, o resto era descartado em vez de reanexado.

Validado com testes automatizados (`pkg/preprocessor/commands_test.go`) e contra arquivos `.prw` / `.ch` reais de um corpus de ~30 mil fontes Protheus (legado 811R4 + versão 12.1.2510 atual) cedido pelo usuário para esta investigação — usado só localmente para validação, não redistribuído. Sem regressões: `make test` continua 30/30, `go vet` e os demais pacotes seguem limpos, cross-compile OK em linux/windows/darwin (amd64+arm64).

[1.6.0] — 2026-07-09

`tests/real_protheus_test.prw` totalmente resolvido

O dump de 3785 linhas de código Protheus real usado como fixture de estresse — que tinha uma falha de parser documentada como conhecida desde antes desta série de correções — agora **compila e interpreta sem nenhum erro** (`advplc check` e `advplc run`, ambos saem limpo). Oito bugs reais e distintos encontrados e corrigidos por biseccção binária (truncar o fonte progressivamente até isolar a menor entrada que ainda reproduz o erro), além dos cinco já corrigidos na versão anterior:

- `++nome` — incremento **prefixo** (só o pós-fixado `nome++` estava implementado).
- `@ ... LISTBOX ... FIELDS HEADER a,b,c ... ON DBLCLICK expr NOScroll OF window PIXEL` — cláusulas do LISTBOX (`FIELDS`, `HEADER`, `ON <evento> <expr>`, `NOScroll`) não reconhecidas.
- `@ y,x BUTTON var PROMPT "texto" ...` — cláusula `PROMPT` do BUTTON não reconhecida.
- `IF ( aArray[ i , j ] )` — o lookahead que desambigua bloco `If` de `IF(cond,then,else)` (adicionado na correção anterior) contava a vírgula de um índice multi-dimensional `[i,j]` como se fosse a vírgula de topo do `IF(...)`, tratando incorretamente todo `If` cuja condição usa um array 2D como a forma de chamada.
- `f(aArray[i] := valor, ...)` — atribuição como argumento de função quando o alvo não é um identificador simples (só `ident := valor` virava atribuição; `array[i] := valor` ficava com o `:=` sobrando).
- `@ y,x RADIO var VAR nVar ITEMS v1,v2,...` — cláusula `ITEMS` do RADIO não reconhecida.
- `Do Case ... End Case` — só `EndCase` (uma palavra) era aceito como fechamento; `End Case` (duas palavras, forma clássica do Clipper) não.
- `FindFunction("Nome")` — nativa ausente (usada no Protheus real para checar a existência de funções opcionais/AddOn antes de chamá-las). Implementada: verifica natives registradas e funções do bytecode (com/sem prefixo `U_`).

Sem regressões: `make test` agora dá **30/30** fixtures (antes eram 29/30, com esta sendo a única falha conhecida); `go vet ./...` e os testes de `pkg/llm` / `pkg/mcp` / `cmd/advplc` continuam limpos; cross-compile OK em linux/windows/darwin (amd64+arm64).

[1.5.0] — 2026-07-09

Servidor MCP nativo (classe `MCPServer`)

O AdvPP agora fala **MCP (Model Context Protocol)** de verdade — ao contrário do suporte a REST (`WSRESTFUL` / `@Get` / `@Post`), que hoje é só sintaxe reconhecida e descartada (sem servidor HTTP nem despacho real), a classe `MCPServer` sobe um servidor **funcional**: JSON-RPC 2.0 sobre stdio, expondo funções AdvPL/TLPP como "tools" que qualquer cliente MCP (Claude, outros agentes) pode listar e chamar.

- `pkg/mcp`: núcleo do protocolo em Go puro (sem CGO, sem dependências externas) — `initialize`, `notifications/initialized`, `tools/list`, `tools/call`, `ping`; transporte stdio com uma mensagem JSON por linha.
- Classe `MCPServer` (`pkg/vm/mcp_native.go`):


```
oMCP := MCPServer().New("meu-servidor", "1.0.0")
oMCP:AddTool("soma", "Soma dois números", ;
    '{"type":"object","properties":{"a":{"type":"number"},"b":{"type":"number"},"required":["a","b"]}', ;
    "ToolSoma")
oMCP:Serve() // bloqueia lendo/escrevendo em stdin/stdout

User Function ToolSoma(oArgs)
Return cValToChar(oArgs:A + oArgs:B)
```

Cada chamada de tool roda a função registrada numa VM isolada (mesmo mecanismo do `StartJob`) — necessário porque `Serve()` já está no meio da execução da VM principal quando uma `tools/call` chega; chamar a função direto na mesma VM corromperia a pilha de chamadas em andamento (bug real encontrado e corrigido durante o desenvolvimento).

`Serve()` redireciona `ConOut` /console para stderr automaticamente, para não misturar saída de depuração com as mensagens JSON-RPC no stdout.

- Funciona com `advplc run` normal — não precisa de um comando novo.

Validado com o SDK oficial em Python do MCP (não só testes internos):

handshake `initialize`, `list_tools`, `call_tool` — ver `cmd/advplc/mcp_integration_test.go`.

Correções no parser (encontradas caçando um bug pré-existente)

Investigando uma falha antiga documentada em

`tests/real_protheus_test.prw` (um dump de 3785 linhas de código Protheus real usado como fixture de estresse) via bisecção binária (truncar o fonte progressivamente até isolar a menor entrada que ainda reproduz o erro), foram encontrados e corrigidos cinco bugs reais e distintos de parsing:

1. `&nome.` — o ponto final (terminador explícito clássico do Clipper/AdvPL para a substituição de macro) não era consumido.
2. `&nome.()` / `&(expr)()` — chamada de função cujo nome vem de uma macro; os parênteses da chamada não tinham dono no parser (mesma simplificação já usada para `alias->¯o`: sintaxe consumida, sem modelar a invocação dinâmica — o VM não resolve função por nome em runtime).
3. `@ y,x GROUP var T0 y2,x2 OF window LABEL "... " PIXEL` — a cláusula `GROUP` do comando `@` de diálogo (caixa de agrupamento) usa `T0` e `LABEL` como cláusulas, não reconhecidas antes.
4. `ACTIVATE DIALOG oDlg ON INIT ... CENTERED` — variante clássica (sem o prefixo "MS") do já suportado `ACTIVATE MSDIALOG`.
5. `IF(cond, then, else)` usado como **statement isolado** (resultado descartado) — sempre caía no parser de bloco `If/EndIf`, que não trata `(...)` com vírgulas como chamada. Novo lookahead (`isInlineIfCall`) desambigua da forma bloco `If (cond) ... EndIf`.

`tests/real_protheus_test.prw` avança de ~503 para ~2414 das 3785 linhas antes de esbarrar no próximo gap (não mais um bug de parsing, uma feature genuinamente não implementada) — mantido como falha conhecida documentada no Makefile.

[1.4.0] — 2026-07-09

Motor de inferência LLM embutido (`pkg/llm` + classe `LLM`)

Novo: um motor de inferência para modelos de linguagem quantizados em

I2_S (ternário, formato BitNet), escrito 100% em Go — sem CGO, sem `llama.cpp`, sem dependências de terceiros — compilando e rodando identicamente em Linux, Windows e macOS (amd64 e arm64). Validado **token a token** contra o `llama.cpp` de referência (fork BitNet do projeto) usando o modelo `Falcon3-3B-Instruct-1.58bit`.

- **Parser GGUF** (`pkg/llm/gguf.go`): header, metadados e tensores lidos sob demanda (não carrega o arquivo inteiro em memória de uma vez).
- **Kernel ternário I2_S** (`pkg/llm/i2s.go`): dequantização e matmul contra ativações int8, replicando byte a byte o algoritmo de `ggml-quants.c`.
- **SIMD AVX2** (`pkg/llm/simd_amd64.s`, amd64): o dot-product ternário em assembly Go (VPMADDUBSW/VPSRLW), com detecção de CPU em runtime via CPUID e fallback automático para o caminho escalar em CPUs sem AVX2 — ou em qualquer arquitetura fora de amd64 (arm64 usa o escalar puro já validado; sem assembly não testável nesta arquitetura).
- **Forward pass completo** (`pkg/llm/model.go`): transformer arquitetura "llama" (GQA, RoPE, RMSNorm, FFN SwiGLU) com KV cache incremental.
- **Tokenizer BPE** (`pkg/llm/tokenizer.go`): byte-level estilo GPT-2, usando o vocabulário/merges já embutidos no próprio GGUF.
- **Amostragem** (`pkg/llm/sampling.go`): greedy, temperatura, top-k, top-p.
- **Classe AdvPL/TLPP LLM** (`pkg/vm/llm_native.go`): expõe o motor como native, no mesmo padrão de `FWMBrowse` / `MsDialog`:

```
oLLM := LLM():New("/caminho/modelo-i2_s.gguf")
cTexto := oLLM:Generate("The capital of France is", 6, 0) // prompt, nMaxTokens, nTemperatura
aTokens := oLLM:Tokenize("algum texto")
cTexto := oLLM:Decode(aTokens)
oLLM:Close()
```

Desempenho (Falcon3-3B-1.58bit, 8 núcleos): ~5s/token com paralelização por goroutines (matmul e atenção por faixa de linhas/cabeças) + caminho rápido sem checagem de limite para blocos ternários completos; AVX2 reduz mais ~1.6x sobre isso em amd64.

Limitações conhecidas: só arquitetura GGUF `"llama"` com pesos I2_S (não `bitnet-b1.58` com as normas extras "SubLN"); pré-tokenizador simplificado (não replica o split dígito-a-dígito específico da Falcon3 — só afeta números com mais de um dígito); sem streaming token-a-token na classe `LLM` (bloqueia até `Generate()` terminar); sem suporte a outras quantizações (Q4_K, Q6_K etc.) nem outras arquiteturas.

[1.3.0] — 2026-07-09

Renderer web (`advplc serve`) — fases 1 a 4

Novo modo de execução: o programa AdvPL/TLPP roda no servidor (mesma VM, mesmo `ADVPP.db`) e a interface é renderizada no browser. Basta o binário `advplc` e um navegador — sem SmartClient, sem executável gráfico.

- **Fase 1 — console e diálogos:** `advplc serve <fonte> [--port N]`. `ConOut` é transmitido em tempo real; `MsgInfo` / `MsgStop` / `MsgAlert` / `MsgYesNo` bloqueiam a execução até a resposta do usuário no browser. Protocolo SSE + POST (stdlib pura, sem WebSocket). Cada aba/recarga é uma sessão com VM isolada e conexão própria ao banco.
- **Fase 2 — MVC → PO-UI:** frontend **PO-UI/Angular** (TOTVS) embutido no binário via `embed.FS`. `FWMBrowse():New()` + `SetAlias("SA1")` + `Activate()` renderiza um `po-table` com colunas e títulos vindos do dicionário **SX3** do `ADVPP.db`; Incluir/Editar abrem um

- `po-dynamic-form` gerado do dicionário; exclusão é soft-delete padrão Protheus (`D_E_L_E_T_='*'`). CRUD persistido no SQLite.
- Fase 3 — hot reload:** `advplc serve <fonte> --watch` recompila a cada alteração do fonte e recarrega as sessões do browser automaticamente; erro de compilação aparece no console do browser.
- Fase 4 — MSDIALOG legado:** `DEFINE MSDIALOG` + `@ linha,coluna SAY/GET/BUTTON` + `ACTIVATE MSDIALOG` viram um modal PO-UI por heurística de grade (controles agrupados em linhas por proximidade de `y`). O valor digitado nos `GET` s **escreve de volta nas variáveis** do programa (novo `FunctionInfo.LocalNames` no bytecode). `ACTION` de botão executa em VM isolada: `VALID / WHEN / ACTION` agora são lazy (embrulhados em codeblock, como o `#xcommand` real do Protheus).

Infra

- `webui_port` na configuração compartilhada (`~/advpp/advpp_config.json`); precedência: `--port` → `config` → 8080. Diretiva do projeto: toda nova configuração entra na Config compartilhada para futura edição via AdvCfg.
- Novo alvo `make web` : recompila o frontend PO-UI e embute em `pkg/webui/dist` (o dist é versionado — `go build` funciona sem Node).
- `SQLiteEngine` ganhou `QueryRows / Exec` (interface `vm.SQLiteEngine`).
- Fixtures novos: `tests/webui_test.prw` , `tests/mvc_browse_test.prw` , `tests/msdialog_test.prw` .

Limitações conhecidas (fase 4)

- Codeblocks deste runtime não capturam variáveis locais: `ACTION {|| oDlg:End()}` não fecha o diálogo — por isso, qualquer clique de botão fecha o diálogo após executar o `ACTION` .
- `VALID` ainda não dispara round-trip por campo (planejado).

[1.2.0] — 2026-07-08

Multi-thread

- `StartJob(cFunc, cEnv, lWait, params...)` implementado no runtime: executa a função em uma VM isolada (semântica de work process do Protheus). Com `lWait=.F.` roda em goroutine e o processo aguarda os jobs pendentes antes de encerrar; cada job abre a própria conexão ao banco SQLite (WAL).
- `FWGridProcess` implementada conforme a documentação TDN: `New` , `SetThreadGrid / SetMaxThreadGrid` (pool de threads com backpressure), `CallExecute` (cada unidade em VM isolada com conexão própria), `Activate / Execute` , `StopExecute` , `IsFinished` , `SetAbort` , `SetAfterExecute` , `meters ( SetMeters / SetMaxMeter / SetIncMeter )` e `SaveLog / GetLastLog` . Sem a interface gráfica de configuração (runtime headless).
- `advplc check` **paralelo**: aceita múltiplos arquivos (antes das flags) e verifica com 1 worker por CPU, com resumo `ok/failed` .

Performance

- Lexer ~95× mais rápido em arquivos grandes:** `tryDotLiteral` fazia `ToUpper` de todo o fonte restante a cada caractere `.` ($O(n^2)$). Fonte real de 574KB: 9,1s → 0,095s. Corpus de 300 fontes reais do Protheus 12.1.2510 verificado em ~1,2s.

Compatibilidade de linguagem

- Lexer tolera backtick solto fora de strings (typo presente em fontes reais da TOTVS aceito pelo compilador Protheus).

[1.1.x] — 2026-07-08

Banco de dados unificado

- **Banco padrão renomeado para** `~/.advpp/ADVPP.db` (era `./data/advpl_dictionary.db`, caminho relativo que quebrava fora do diretório do projeto).
- **Resolver único de caminho** (`shared.ResolveDatabasePath`) usado por todas as ferramentas: flag explícita → variável `ADVPP_DB` → config `~/.advpp/advpp_config.json` → legado `./data/` → padrão absoluto.
- **Ponto único de abertura** (`shared.OpenSQLite`) com pragmas `WAL`, `busy_timeout` e `foreign_keys` para todas as ferramentas.
- **VM conectado ao banco compartilhado:** `--db-path / ADVPP_DB` agora funcionam de fato no `advplc run / exec` (antes eram parseados e ignorados); a IDE também conecta o VM ao mesmo banco.
- Corrigido schema do dicionário: criação do zero falhava por colunas ausentes em SX2 (`X2_NOMEUSR / X2_MODULO / X2_TIPO / X2_DESCRIC`) e SX5 (`X5_TIPO / X5_TAMANHO / X5_DECIMAL`).
- Corrigida a heurística `banco.db/tabela` do driver SQLite, que quebrava qualquer caminho absoluto (agora só ativa quando o caminho não existe em disco; aceita `/` e `\`).

Portabilidade (Linux / Windows 64 / macOS)

- **Driver SQLite trocado para** `modernc.org/sqlite` (100% Go, sem CGO): o CLI cross-compila estaticamente para linux/windows/darwin, amd64 e arm64.
- **Removida a dependência do** `iconv` **externo:** conversão CP-1252 → UTF-8 é feita por conversor interno 100% Go, idêntico nas 3 plataformas.
- `go.sum` versionado (estava incorretamente no `.gitignore`).

Build, empacotamento e release

- **Makefile**: `make build` (4 ferramentas), `make test` (fixtures), `make cross` (CLI para 5 alvos), `make package VERSION=x.y.z` (pacotes em `dist/`), `make release VERSION=x.y.z` (tag + CI).
- **GitHub Actions** (`.github/workflows/release.yml`): a cada tag `v*`, builds nativos em Linux, Windows e macOS (incluindo as GUIs Fyne) e publicação automática dos pacotes `.tar.gz / .zip / .deb` na Release.
- `advplc version` mostra a versão embutida no build.
- Corrigido `.gitignore` que ignorava o diretório `cmd/advpp-ide` (o fonte da IDE não estava no repositório).

[1.0.0]

- Versão inicial: compilador (lexer, preprocessador, parser, codegen), VM com natives, MVC, UI Fyne, ferramentas advcfg/adveditor/advpp-ide.